New Era University
College of Informatics and Computing Studies
Computer Science Department

NEUDev: An Intelligent Coding Assessment Platform for New Era University

An undergraduate thesis presented to the Faculty of New Era University,
College of Informatics and Computing Studies,
Computer Science Department
New Era, Quezon City

In Partial Fulfillment of the Requirements for the Degree in
Bachelor of Science in Computer Science

Achero, Noriel R.
Celajes, Aeron Red R.
Condada, Hannah L.
Enaje, Erikka Marielle M.
Manliguez, Angelica Mae A.

MAY 2025
APPROVAL SHEET

The thesis entitled **NEUDev An Intelligent Coding Assessment Platform for New Era University**, prepared and submitted by **Noriel R. Achero, Aeron Red R. Celajes, Hannah L. Condada, Erikka Marielle M. Enaje, Angelica Mae A. Manliguez,** in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science has been examined and recommended for acceptance and approval for the oral examination.

Dr. MARC P. LAURETA
Thesis Adviser

Approved by the Oral Examination with a grade of _____ on ____/____/____.

Prof. IRYSH PAULO R. TIPAY
Committee Chair

Prof. ELIJIAH L. QUILATAN
Member

Prof. EDILBERTO SIMBULAN JR.
Member

Accepted as a partial requirement for the degree in Bachelor of Science in Computer Science.

Engr. JEREMIAS C. ESPERANZA
Program Head

Prof. AUDREY LYLE D. DIEGO
Dean

ACKNOWLEDGEMENT

With boundless love and deep appreciation, we would like to extend our heartfelt gratitude to all the individuals who supported and contributed to the success of this study.

First and foremost, we give our utmost thanks to The Almighty God for His constant guidance, wisdom, and knowledge, and for enlightening us in every step and meeting throughout the course of this research.

We are truly grateful to Dr. Marc P. Laureta, our thesis adviser, for his unwavering support, encouragement, and guidance throughout the course of our study. His advice and dedication served as an inspiration for us to finish this paper.

We also want to extend our heartfelt thanks to our classmates and fellow 4 BSCS-1 members. Their teamwork, constant encouragement, and friendship helped us overcome the challenges we encountered along the way.

To our parents, thank you for endless patience, understanding, and support emotionally, morally, and financially. Your sacrifices made it possible for us to accomplish this study. Above all, we offer our deepest gratitude and praise to God, who made everything possible.

DEDICATION

This work is lovingly dedicated to our families, whose unwavering support and understanding gave us strength throughout this journey. To our parents, thank you for your endless sacrifices, guidance, and unconditional love. Your belief in us has been our greatest motivation. To our siblings, thank you for your patience, encouragement, and for cheering us on, even in silence. To our dear friends and classmates, thank you for the camaraderie, shared struggles, late-night study sessions, and the constant support that made this experience more meaningful and memorable. And most importantly, to Almighty God, thank You for the wisdom, strength, and grace that allowed us to persevere and succeed.

- N.R. Achero
- A.R.R. Celajes
- H.L. Condada
- E.M.M. Enaje
- A.M.A. Manliguez

ABSTRACT

Title	:	**NEUDev: An Intelligent Coding Assessment Platform for New Era University**
Authors	:	Noriel R. Achero Aeron Red R. Celajes Hannah L. Condada Erikka Marielle M. Enaje Angelica Mae A. Manliguez
Adviser	:	Dr. Marc P. Laureta
School	:	New Era University

Keywords: *Coding Assessment Platform, Artificial Intelligence, AI-generated code detection, Code similarity analysis, Academic integrity and anti-cheating mechanisms, CodeSpy, Rubrics*

This study introduces NEUDev, an intelligent coding assessment platform specifically designed to modernize programming evaluation at New Era University by embedding AI-driven integrity checks and automated scoring. Recent academic trends have revealed a critical gap in programming education: the increasing reliance of students on AI tools like ChatGPT and Copilot to generate code, often bypassing essential learning and problem-solving processes. Traditional plagiarism detectors are also largely ineffective at identifying AI-generated content, posing new challenges to academic integrity. Apart from these, traditional and manual assessment evaluations are also time-consuming and tedious while coding standards are frequently inconsistent

between teachers, thus failing to fairly evaluate advanced programming cognitive abilities of students such as problem-solving, debugging, and code optimization.

To address these issues, the platform integrates several solutions: are to (1) provide an in-browser execution environment for Python, Java, and C#; (2) integrate CodeSpy's AI Generated Code detector to flag AI Generated Code submissions; (3) incorporate DOLOS for similarity analysis to uncover potential plagiarism among students' submissions; (4) employ a test-case-based scoring framework combined with a detailed rubric to deliver scores and feedback; and (5) validate the system's reliability through rubric-based evaluation metrics (Precision, Recall, F1-Score, and overall Accuracy).

To support evaluation, a synthetic dataset of 90 console-based programs (30 per language) was generated via the OpenAI API. Each record including programming language, question, source code, allotted time, and actual time taken and they are manually annotated across the five rubric criteria by human. In the evaluation, NEUDev's rubric system achieved a high accuracy in the different criteria.

TABLE OF CONTENTS

PRELIMINARIES **PAGE**

APPROVAL SHEET ii
ACKNOWLEDGEMENT iii
DEDICATION iv
ABSTRACT v
TABLES OF CONTENTS vii
LIST OF FIGURES x
LIST OF PSEUDOCODE xii
LIST OF TABLES xiii
LIST OF EQUATIONS xiv

CHAPTER 1 INTRODUCTION 1

Background of the Study 1
Objectives of the Study 2
Significance of the Study 3
Scope and Limitation 4

CHAPTER 2 REVIEW OF RELATED LITERATURE 6

Online Coding Assessment Platforms 6
CodeChum 6
LeetCode 7
HackerRank 8
ACCE 9
Autograder 9
Boss 10
ProgEdu 10
GradeBot 11
CodeOcean and CodeHarbor 11
Intelligent Coding 13
Pedagogy Methodologies 13
Online Coding Assessment 16

Online Coding Assessment Platforms 18
Students' Use of LLMs for Programming Assignments 19
Students' Reliance on AI Tools in Programming Education 21
Integrated AI Code Detection Systems 23
Anti-Cheating Mechanisms in Coding Assessments 24
Role of AI in Programming Evaluation and Security 27
AI Code Detection 28
CodeSpy 29
GPTZero 30
Codeleaks 31
ZeroGPT 32
BlueOptima 32
Code Vision 33
GPTSniffer 33
AI Coding Assistant 37
AI Chatbots 37
AI Test Cases Generation 38
AI AutoGrading 38
Synthesis 39
Programming Environment Supporting Python, Java, and C# 39
Detecting AI-Generated Code (CodeSpy) 41
Similarity Checking for Code (DOLOS) 42
Automated Code Evaluation 43
Reliability of AI-Based Grading Evaluation 45

CHAPTER 3 METHODOLOGY 48
Project Design 48
Conceptual Framework 51
FrontEnd Layer 54
BackEnd Layer 56
Code Execution Engine 59

Database 61
API Tools Integrated 61

CHAPTER 4 RESULTS AND DISCUSSION **64**
NEUDev's Code Execution 65
CodeSpy AI Code Detection 68
DOLOS Similarity System 71
Test Case and Rubrics System 75
NEUDEV's Rubrics 76
Scoring Formulas 79
Thresholds 83
System Usage 95

CHAPTER 5 CONCLUSION AND RECOMMENDATION **99**
Conclusion 99
Recommendation 102

REFERENCES **105**
ONLINE REFERENCES .....
APPENDICES

LIST OF FIGURES

FIGURES NO.	TITLE	PAGE
FIGURE 3.1	*Project Design*	**48**
FIGURE 3.2	*NEUDev Learning-Integrity Feeback Loop (LIFL) Framework*	**51**
FIGURE 3.3	*Test Data*	**63**
FIGURE 4.1	*Python Execution in Sandbox*	**66**
FIGURE 4.2	*Java Execution in Sandbox*	**67**
FIGURE 4.3	*C# Execution in Sandbox*	**67**
FIGURE 4.4	*Chat GPT Generated Submitted Code*	**69**
FIGURE 4.5	*AI Code Prediction in the Teacher's Record of Student Submission Table*	**69**
FIGURE 4.6	*Student Human Submitted Code*	**70**
FIGURE 4.7	*Human Prediction in the Teacher's Record of Student Submission Table*	**70**
FIGURE 4.8	*Overview of the statistic of the similarity in a python activity*	**72**
FIGURE 4.9	*Pairs of similarity in code submissions*	**73**
FIGURE 5.1	*Highlights of code in pairs of similarity in two code submissions*	**74**
FIGURE 5.2	*Gemini API*	**80**
FIGURE 5.3	*Test Case Scoring*	**82**
FIGURE 5.4	*Rubric Scoring*	**82**
FIGURE 5.5	*NEUDev_BE*	**88**
FIGURE 5.6	*NEUDev_Compiler*	**89**
FIGURE 5.7	*Google Forms Summary of Student Feedback on NEUDev Platform Experience*	**90**
FIGURE 5.8	*Student Ratings on the Usability of the NEUDev Integrated Development Environment (IDE)*	**91**
FIGURE 5.9	*Student Ratings on the Accuracy and Usefulness of AI Rubrics in NEUDev*	**92**

FIGURE 6.1 *Student Ratings on the Class Management Features of NEUDev* **92**

FIGURE 6.2 *Student Ratings on the Effectiveness of the Activities Section in NEUDev* **93**

FIGURE 6.3 *Student Ratings on the Usefulness of Test Cases in NEUDev* **94**

FIGURE 6.4 *Student Ratings on the Overall Assessment Page Environment in NEUDev* **94**

FIGURE 6.5 *NEUDev System Usage (May 14, 2025)* **96**

FIGURE 6.6 *NEUDev System Usage (May 15, 2025)* **97**

LIST OF PSEUDOCODE

PSEUDOCODES NO.	TITLE	PAGE
TABLE 1.1	*NEUDev's Code Execution*	**65**
TABLE 1.2	*CodeSpy AI Code Detection*	**68**
TABLE 1.3	*DOLOS Similarity System*	**72**
TABLE 1.4	*Test Case and Rubrics System*	**76**

LIST OF TABLES

TABLES NO.	TITLE	PAGE
TABLE 2.1	Comparison of AI Code Detection	**34**
TABLE 3.1	*Frontend Technologies Used*	**54**
TABLE 3.2	*Backend Technologies Used*	**57**
TABLE 3.3	*Code Engine Integration*	**60**
TABLE 3.4	*NEUDev's Rubics*	**77**
TABLE 4.1	*Thresholds*	**86**
TABLE 4.2	*Total Results*	**85**
TABLE 4.3	*HTTP Details*	**87**
TABLE 4.4	*Execution*	**87**
TABLE 5.2	Comparison of Human and AI Grading Accuracy Across Performance Levels	**99**

LIST OF EQUATIONS

EQUATIONS NO.	TITLE	PAGE
EQUATION 4.1	*Points per test case*	**81**
EQUATION 4.2	*Remaining points after run penalties*	**81**
EQUATION 4.3	*Fraction of tests passed*	**82**

EQUATION 4.4 *Score for one item* **82**

CHAPTER 1

INTRODUCTION

This chapter provides an introduction to the study, offering insights into the study's background, and outlining the objectives, significance, scope, and limitations of the research.

Background of the Study

Traditional methods of coding assessment and grading in educational institutions, particularly in programming courses, provide numerous challenges. One challenge is the reliance on manual grading, which is time-consuming and often inconsistent due to the bias of human assessment. Traditional assessments typically include static and standardized paper-based tests, which do not fully capture the dynamic and iterative nature of programming. As a result, these methods fail to properly evaluate advanced cognitive abilities such as problem-solving, debugging, and code optimization which are crucial in computational thinking and programming education (Kong & Liu, 2020). Modern education requires flexible, efficient assessment methods that provide immediate feedback. Automated code assessment systems address this problem by automatically grading code and providing detailed error feedback (Combéfis, 2022).

Advances in AI-powered coding assistants and LLMs, notably GitHub Copilot, Gemini, Claude, and ChatGPT are transforming how students approach programming assignments. These tools can automatically generate code or help

debug and explain programs based on natural-language prompts. Ever since AI became prominent, they've quickly entered classrooms and assessments. Researchers are now examining how this 'AI assistance' affects students' learning habits and foundational skills in programming. While these tools offer convenience and potential learning support, a lot of studies highlight the negative effects on learning from the formation of bad coding habits to the for problem-solving skills. (Copilot in 2021, ChatGPT in late 2022),

Directly addressing these gaps, the researchers created NEUDev, an intelligent coding assessment platform developed and designed for New Era University which makes the scoring and evaluation of coding assessments better and efficient by using advanced technologies to reduce the burden of manual grading and scoring of students' coding assessments. The system is built with modern web technologies, NEUDev is developed using ReactJS and Laravel Framework that adheres to the Model-View-Controller design pattern that ensures efficiency, scalability, and maintainability. The platform also incorporates the Railway for online hosting compiling and interpreting the programs of Java, Python, and C#. The researchers also integrated CodeSpy for detecting AI code submissions, and Dolos to detect similarity between submitted codes by students.

Objectives of the Study

This study aims to develop NEUDev that will, specifically it seeks solutions with the following objectives:

1. To provide a programming environment where students can execute the selected programming languages: Python, Java, and C#

2. To integrate CodeSpy's AI code detection capability into the NEUDev platform for identifying AI-generated code submission.

3. To integrate DOLOS into NEUDev for detecting code similarity among student submissions and assisting instructors in identifying potential cases of plagiarism.

4. To implement a code evaluation system that uses test-case-based scoring framework and rubric system to provide students with quantifiable scores and targeted feedback on their code submissions.

5. To evaluate the overall system's rubric-based approach and assess the accuracy and reliability of the AI's automated grading using F1 Score, Accuracy, Precision, and Recall.

6. To evaluate the overall system's rubric-based approach and assess the accuracy and reliability of the AI's automated grading.

Significance of the Study

The researchers have identified the key beneficiaries who are estimated to benefit most from implementation of the proposed project, which in turn includes individuals and groups who will directly or indirectly benefit from the features, functionalities and results of the system. The list below shows the main beneficiary as well as the ways in which they will benefit.

Institutions. The institutions will benefit from improved quality of programming education, increased educator productivity, and the potential to attract new students by offering an advanced, technology-driven learning environment.

Educators. The platform will reduce the manual grading burden and ensure consistent, objective assessment of student work. It will automate routine grading tasks and provide insights into students' code strengths and weaknesses.

Students. They will enhance their understanding of programming concepts and improve their skills in Java, Python, and C# through advanced. The platform fosters iterative learning by providing continuous feedback, helping students to identify and correct their mistakes effectively after assessments.

Future Researchers. This study will provide valuable insights and methodologies for developing new platforms focused on Future researchers that can build upon this work to further advance the field of automated code review and educational technology. The proposed project can be open source to future researchers who can improve the system and add new features including AI systems.

Scope and Limitation

The proposed study aims to analyze and evaluate source code submissions of student developers in the NEUDev platform, namely three (3) of the most prominent programming languages, Java, Python, and C#. These

submissions will be assessed in different paradigms utilizing the NEUDev platform, which integrates Codespy for AI-generated code detection and Dolos for code similarity among students' submissions

The NEUDev platform is developed using ReactJS and the Laravel Framework to ensure a responsive user interface and efficient backend operations. The platform is hosted on Railway, providing a flexible deployment for users.

However, the study has certain limitations to consider. It focuses only on Java, Python, and C# programming languages and is limited to the educational context of New Era University. Results may vary when applied in other institutions or fields. Additionally, the accuracy of code similarity and AI detection results may depend on the quality and diversity of the input data. The study does not work in real-time; it simply processes pre-written source codes as an input to the model. Lastly, the NEUDev platform's performance and scalability are technically constrained by hardware resources and deployment of the system to the serve.

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter presents the studies gathered and reviewed to establish a solid foundation for the present study. It includes the Review of Related Literature and Studies, Synthesis and Justification, Conceptual Framework of the Study, and the Definition of Terms.

Online Coding Assessments Platform

Online Coding assessment platforms are digital tools made to help students learn and practice programming while also getting evaluated on their skills. These platforms usually offer a space where learners can write code, solve exercises, and receive immediate feedback–all within a browser, no extra software needed. They've become especially valuable in recent years, particularly during the sudden shift to online learning brought about by the pandemic. With physical classes on hold, many students struggled to keep up with programming subjects on their own. According to (Zinovieva, 2021), a large number of students faced difficulties adjusting to remote setups, and that's where online coding platforms really stepped in making learning more engaging through hands-on tasks, tutorials, and even gamified exercises.

CodeChum

CodeChum is an online web-based Integrated Development Environment (IDE) developed to assist professors and instructors in teaching programming courses in order to aid students in learning programming. Key features of this

platform are its ability to automate the creation and checking of coding exercises. It also provides instant feedback where the student's performance is included in the dashboard to monitor their performance.

The study highlights the growing demand for software engineers and the challenges faced by professors and students alike in programming courses. CodeChum aims to address these challenges by providing a platform that is user-friendly, accessible, interactive, and automated specifically for teaching and learning C programming. In their study, the developers of CodeChum evaluated the platform's effectiveness through surveys and feedback from student users. The result showed a significant improvement in the students programming skills and a higher engagement rate during laboratory exercises. Students reported that the feedback and automated grading features were helpful in understanding their mistakes and correcting them quickly. Professors also noted a reduction in manual checking time, which allowed more focus on teaching logic and concepts.

LeetCode

Leetcode is a platform that helps individuals enhance their skills, expand knowledge, and prepare for technical reviews. It has a well-organized tool that provides structure in order to guide user progress towards their programming career. It provides users a coding platform where they can be competitive as it also conducts coding competitions. Aside from this, discussion forums are also utilized to encourage users in interacting with their fellow programmers wherein it

aids their preparation for interviews. Some benefits of this platform come from its resources on large problem sets, focus on algorithms and data structures, and its

active community. Moreover, LeetCode has also been recognized as a useful platform in coding assessments due to its structured challenges and time-bound environments. Although primarily designed for practice, many educators and companies use it to simulate real-world coding tests and evaluate technical competence. Studies suggest that its problems, categorized by difficulty and topic, can serve as reliable indicators of programming proficiency (The Forage, 2023). This makes LeetCode not only a training ground but also a potential tool for objective assessment.

HackerRank

HackerRank is a programming practice platform that provides competitive challenges and tests for learning skills and preparing for technology careers Girardin (2023). It is an excellent tool for honing an individual's proficiency in programming languages. Aside from this, employers are also empowered in analyzing their candidates' coding capabilities before investing their time in interviews.

HackerRank is not only an employer–employee assessment tool, it can also be utilized by students and learners who simply want to challenge their programming skills such as in learning new algorithms, data structures, and problem-solving techniques.

This platform has the ability to simulate a real coding interview experience which helps students enhance their communication skills and problem-solving skills even under pressure.

ACCE

The Automated Coding Composition Evaluator (ACCE) uses static analysis and clustering to evaluate programming code. ACCE converts code into abstract syntax trees (ASTs) and uses unsupervised learning to find common patterns and mistakes. This system automates grading and provides detailed feedback on coding style, helping students improve their programming habits and code readability. Visual representations in ACCE show clusters of similar coding styles, helping students see where they stand compared to their peers and identify areas for improvement (Rogers, 2022).

Autograder

AUTOGRADER, an automated grading tool, uses formal semantics and symbolic execution. AUTOGRADER compares the execution paths of student code to a reference implementation to find semantic differences. This method ensures high accuracy by catching errors that traditional test-based systems might miss. AUTOGRADER generates counterexamples for incorrect submissions, providing valuable feedback to help students understand and correct their mistakes (Liu, 2019).

Despite its advantages, AUTOGRADER has limitations. It requires a correct reference implementation, which can be hard to create. Symbolic

execution and formal semantics are computationally intensive, leading to longer processing times, especially for complex code. The system may also struggle with programs that have many different execution paths, potentially missing errors or producing false positives.

Boss

An online submission and assessment system, developed by (Joy, Griffiths, & Boyatt, 2022) from the University of Warwick, is an educational technology tool designed to streamline programming assignment assessments. It automates submission collection, correctness and code quality testing, plagiarism detection, and feedback facilitation, focusing on Java and supporting both web based and standalone Java application clients. BOSS uses methods like file based and JUnit testing for Java to verify correctness and evaluates code quality through various metrics. It features a secure architecture with separate servers for students and staff, a sandbox for safe code execution, and SQL databases with the Core Soft schema for data management. However, BOSS has limitations, such as difficulties with deep learning or complex tasks, complex and time-consuming test setups, mixed usability feedback for the staff interface, challenges in integrating with external systems like the Student Record System (SRS), and potential performance issues near submission deadlines.

ProgEdu

ProgEdu is an automated assessment platform for programming courses, developed by (Chen, 2022) to address the challenges in managing and evaluating programming assignments. It integrates student management,

assignment management, and a visual analysis dashboard to reduce workload, provide real time submission status, and help students check their submissions against requirements immediately. ProgEdu uses Git for version control, allowing direct submission from IDEs to repositories like GitHub, and integrates Jenkins for

automatic, including compilation checks, plagiarism detection, and unit tests. The platform features batch registration, assignment management GUI, and visualized dashboards for data analysis and progress tracking. However, it has limitations such as manual account management, complex infrastructure setup, and a steep learning curve for Git. While it helps track submission progress, it may not fully capture all aspects of student learning behaviors and assignment challenges.

GradeBot

GradeBot, developed by (Colton, 2022) is a computer programming lab assignment evaluator that operates automatically. It is intended to facilitate the rapid acquisition of programming skills by students in an introductory course by offering a greater number of practice problems at a higher frequency, while simultaneously reducing the grading burden on professors.

Several advantages of Gradebot are its ability to increase assignment frequency, provide immediate feedback, and hold a standardized and unbiased evaluation.

It saves an instructors' time, utilizes distance education, supports larger class sizes, improves student learning, and detects potential cheating instances; On the other hand, it is generally unsuitable for graphical programs with interactive outputs and test cases which could lead to problems addressing student collaboration and exam performance.

CodeOcean and CodeHarbor

A practical programming assignment support in MOOCs and educational environments; CodeOcean, developed by (Serth, 2022) provides an automated grading system across multiple languages. It has a user-friendly web based integrated development environment (IDE). The platform provides instant feedback along with annotations and allows the user to execute their program without limit before submitting. It also enables integration with several Learning Management Systems through the Learning Tools Interoperability and helps professors customize study groups. However, it had some issues when it came to managing the highly high stream of users.

Some restrictions are faced when it comes to advanced static or pair programming. CodeHarbor, on the other hand, functions as an exchange cooperative platform for professors that has the ability to automatically grade assessments. It facilitates the sharing of auto gradable coding assessments which allows for keeping history versions and grouping of exercises. It allows instructors to collaborate as sharing of test files and metadata is utilized.

One of its provided supports is through its interoperability with other auto graders through the Pro Form A data format. It has weaknesses that come along with its strengths due to its high dependence on a solid community that shares and collaborates on exercises. Instructors with limited technical expertise may also face challenges in its setup and maintenance.CodeOcean and CodeHarbor were built to create a scalable system that will enhance programming education through automated grading and a collaborative platform for both students and professors, allowing them to share educational resources.

Intelligent Coding

In recent years, the concept of intelligent coding has emerged at the intersection of data compression and AI driven optimization. From the “Coding for intelligence” perspective, coding is no longer merely about bit level efficiency but about capturing those intrinsic relationships within data that are most relevant to downstream tasks. (Yang, 2024), formulated this is a Coding for intelligence (CfI) problem, where the goal is to learn a compact representation that preserves task critical information. In this view, an “intelligent” coder is one that adapts its compression strategy to the needs of learning and inference, effectively blurring the line between encoder and model. A concrete instantiation of intelligent coding appears in the design of error-correction codes, where traditional algebraic constructions give way to AI-driven code constructors. Huang (2022), introduce a constructor–evaluator framework in which reinforcement learning and genetic algorithms iteratively refine code parameter Such approaches exemplify intelligent coding by letting a learning agent discover optimal coding structures

tailored to specific channels, decoders, or target error rates rather than relying on expert-handcrafted rules.

Pedagogy Methodologies

The emergence of computer programming in university curriculums prepares students for a future that goes beyond just learning how to code. Its integration has sparked widespread interest due to its potential to improve educational outcomes beyond conventional programming abilities.

Asserts that learning algorithms and coding at a young age is important for information literacy. The reason being that early coding education goes beyond workforce readiness to promote educated and active citizenry. The introduction of Computer Science has become increasingly important in preparing students for future job markets and also for developing critical thinking skills to enable participation in an automated economy. He emphasizes that coding, similar to reading and writing, necessitates both expressive capabilities and problem-solving skills, rendering it a valuable literacy in the 21st century (Bers, 2019).

Research indicates that early educational interventions have a long-lasting impact compared to interventions that are made later in life (Bers, 2019). Although early childhood computer science education is still lacking, early exposure to coding can improve the plasticity of young children's brains, which fosters cognitive and socioemotional development (Bers, 2019). This leads us to the exploration of pedagogy methodologies that are currently utilized in

schools and universities. (Vakaliuk, 2021) conducted a study that explores the implementation of effectiveness of a distance learning course titled “Features of Distance Learning of Cloud Technologies for the Organization Educational Process in Quarantine.” The course was developed in response to the pressing necessity for educators at all levels of education to adjust to remote teaching methods during the COVID19 pandemic.

The authors emphasized the importance of instructors rapidly acquiring the necessary skills to utilize cloud technologies in order to guarantee the quality and continuity of education during periods of quarantine. The training within the course was implemented in three sessions, where each session could contain 1,500 participants. With such numbers, the course experienced a beginning number of 1,029 participants and 816 participants who went ahead to complete it successfully. Those attending the training sessions were teachers from general secondary education, higher education institutions, vocational schools, and any other roles within an educational setting. Participants' knowledge, along with skills on how to apply cloud technologies to distance learning, was dramatically enhanced. It shows clearly that practical tasks are essential and recommended more than theoretical learning because it engages participants more easily.

Hence, it raises the need to find a better way in which coding and programming knowledge could be assessed in the education field. (Charan, 2023) developed a platform by the name "Coding Assessment Portal" for the assessment of the coding ability and technical skills of an individual. The creation

of this platform focused explicitly on recruitment, skill assessment, and education.

This portal was developed aiming to efficiently help in the assessment of coding skills, technical skills, recruitment, and assessment processes while making sure that a fair and standardized evaluation is also offered so that it can be scaled for the evaluation of large numbers of candidates simultaneously. The Coding Assessment Portal by (Charan, 2023) eases the skills evaluation process in coding, as it was designed with the support of a web code editor for different programming languages. Basic functionalities in the form of syntax highlighting, code completion, and debugging tools that are implemented ease this process.

As such, the assessments can easily be customized by defining which programming languages must be used, time constraints, and different difficulty levels are set. The portal system has to be able to automatically assess code on correctness, efficiency, and compliance with guidelines for objectivity and uniformity in grading. This will also promote collaboration and communication more, where the messaging systems of the portal are used so that participants can ask questions, get clarification, or seek feedback from the assessor. Finally, Charan (2023) concluded that the goals set by the assessment portal have indeed been achieved and proved effective in providing a friendly, very efficient, and scalable platform for the overall conduct of coding assessments. They pointed out that it was reaping the most significant benefits for employers and job seekers, making recruitment and skill evaluation more accessible and faster.

Online Coding Assessments

Online Coding assessments have become increasingly vital in programming education, particularly as institutions and educators strive to measure not just theoretical knowledge but also applied skills. These assessments are structured evaluation that test students' ability to write, debug, and optimize code, usually in real-time coding environments. (Misiejuk, Kaliisa, and Scianna, 2024) emphasize that digital tools used for assessment, including AI supported environments, help educators understand students' problem-solving process beyond mere outcomes, highlighting the broader educational value of such assessments ensure that students can apply their learning in practical settings, which is essential in the modem tech industry.

Traditionally, coding assessments were conducted through written exams or in supervised lab sessions where students manually wrote their solutions. However, the shift toward digital and online platforms has introduced more interactive and automated systems. (Zinovieva, 2021) describes how online coding platforms such as Codeforces, HackerRank, and LeetCode have transformed assessment by providing automated grading, instant feedback, and the ability to conduct remote exams. These platforms replicate real-world coding scenarios, offering without the constraints of physical presence. They also support a variety of programming languages and tools, making them versatile for different educational contexts. Despite these advantages, there are notable challenges and limitations. One major benefit is the scalability of online assessments, allowing institutions to accommodate large student populations

efficiently. Students also receive immediate feedback on their submissions, which promotes iterative learning and problem-solving. (Filho, Neto and Santos, 2024), conventional online judge systems often provide only binary feedback (e.g., pass/fail), which limits students' opportunities to understand their mistakes and improve their code quality.

Raises concerns about the reliability of AI based assessments, especially in terms of how consistently systems can interpret and evaluate students' responses. Zinovieva (2021) also notes the difficulty in fully replacing the nuanced, personalized feedback typically given by human instructors, especially when evaluating code style, design patterns or creating solutions. (Misuejuk, 2024),

Online Coding Assessment Platforms

Technological innovations play an ever-increasing role for a country's development and economic growth. Technology touches virtually every part of our daily lives. Thus, there's an increasing demand when it comes to programming skills. Programming has become a key qualification of the 21st century. (Munisamy, 2024), in their systematic review of code, click learn, presents an extensive analysis of widely used online assessments tools in modem programming education. They study popular platforms like Codeforce, HackerRank, and CodeLab, which have been adopted across various educational institutions due to their accessibility and adaptability to different curricula. The platforms typically support multiple programs, integrate timebound

assessment, and provide immediate automated grading enhancing learning efficiency and reducing instructor workload.

Explored an online learning platform tailored for programming education, emphasizing the integration of teaching, learning, and assessment in one cohesive system. Platform featured real-time feedback and formative assessment tools, improving learner engagement and enabling instructors to track individual student progress more effectively. Staubitz (2021) evaluated CodeOcean, a cloudbased coding environment that supports interactive, auto-graded programming exercises. Findings point to the platform's ease of integration and university courses, highlighting features such as sandboxed code execution for academic integrity, instant feedback, and analytics for educators (Robinson and Carroll, 2021). Comparing user experiences across platforms, studies often emphasize intuitiveness, responsiveness, and the quality of feedback. CodeOcean, for instance, was praised for its clean UI and effective integration into eLearning ecosystems (Staubitz, 2021), whereas traditional platforms like Codeforces rely more on community-based feedback and competitive formats (Munisamy, 2024). Research consistently supports the adoption of these platforms, noting improvements in student performance and motivation. Challenges remain, particularly around ensuring assessment integrity and addressing network or technical limitations during high stakes exams.

Students' Use of LLMs for Programming Assignments

The accessibility of Large Language Models (LLMs) such as ChatGPT, Bard, Gemini, etc. greatly revolutionized the way students complete

programming tasks. It was shown in the research work of (Rytilahti and Kaila, 2024) that students with little or no expertise in coding were able to successfully complete programming courses with the help of AI assistance. Through a simulation of a student with no programming knowledge, the researchers established that the student was still capable of scoring 63.4% of the course points by merely copy pasting AI suggested solutions (Rytilahti & Kaila, 2024).

The test was performed with a beginner level of a Python programming course through the Vile platform. Students typically code to complete exercises, with the system grading their responses automatically. Two methods were tested through simulation: one where there was no programming proficiency at all, and one where the student was presumed to have some skill to repair mistakes with the aid of feedback (Rytilahti & Kaila, 2024). Results indicated that with ChatGPT assistance, students were able to complete from 75.9% to 98.6% of the programming exercises correctly, depending on the method (Rytilahti & Kaila, 2024). ChatGPT, including versions of GPT3.5 and GPT4, was also capable of passing several versions of the exams that are part of the course. This is a severe academic integrity concern, particularly for fully online or self-paced courses, where the aid of AI can be easily hidden (Rytilahti & Kaila, 2024). The kind of errors committed by ChatGPT generated answers were largely trivial, like output format errors or minor module misuse.

Most of the errors were neither logical nor conceptual, they were such that even an elementary student that has little knowledge of programming can easily correct them (Rytilahti & Kaila, 2024). It points towards a major risk where

students may not comprehend coding concepts fully but may pass exams using a lot of AI.

The study of (Rytilahti and Kaila, 2024), emphasized that existing AI detectors are poor at detecting AI generated content with certainty, particularly for short code submissions common for programming tasks. It is determined that although techniques such as modifying question types or giving confusing tasks may delay the misuse of LLMs, these are unsustainable long-term measures. Because LLMs are always enhancing, efforts should be directed towards educating students to use AI tools ethically as opposed to trying to ban the applications of the same (Rytilahti & Kaila, 2024).

Students' Reliance on AI Tools in Programming Education

A study by (Amoozadeh, 2024) identified the new problem of students using AI for programming tasks. Their research was a mixed methods case study of how CS1 students interacted with ChatGPT while performing the tasks of Python programming. They learned that nearly a third of the students provided the complete programming problem description to ChatGPT without even trying to solve the assignment themselves first. This poses a significant concern of students avoiding fundamental problem-solving procedures, which are indispensable for true skill acquisition. The research also determined that although Generative AI applications such as ChatGPT are capable of immediate support as well as augmenting accessibility to code explanations, they tend to inadvertently promote poor help seeking behavior from novice developers (Amoozadeh, 2024). Rather than using AI strategically for clarification requests

or debugging support, some of the students had a tendency to over rely upon AI for full-fledged answers. Very few of the students were making attempts to validate the answers they were getting from ChatGPT, which means they lacked reflective as well as critical evaluation.

The effectiveness of students' programming results was neither directly correlated with the number of times they used AI. Instead, it was more about how the students interacted with the AI. If they employed systematic step-by-step paraphrasing as well as hybrid methods (mixing independent coding and AI assistance), they performed more successfully than students who only submitted complete problem descriptions. There is a lack in helping students use AI tools responsibly, thereby fostering lack of critical thinking, self-efficacy, and independent problem-solving abilities during programming assessments. It also calls for the need to design an educational platform that encourages students to break down problems and actively engage in the learning process rather than passively consuming AI-generated outputs. Another study that addresses the matter of AI dependency by university students was conducted by (Li, Jiang, Ma, Huang, and Wu, 2024). This research focused on how growing utilization of ChatGPT in students' academic settings has led to students relying heavily on AI applications for academic work. While ChatGPT provides assistance with the tasks of translation, writing generation, as well as even emotional support, it presents a number of unforeseen effects that negatively affect students' learning (Li, 2024).

The researchers determined that overreliance on ChatGPT promotes a narrowing of students' learning breadth and depth. Students mostly take advantage of AI-generated answers just to complete assignment tasks without acquiring a profound knowledge of the subject matter. This practice thwarts their chance to establish a comprehensive and substantial knowledge base (Li, 2024). Rather than practicing intensive studies, students typically end up embracing shallow learning practices, which can negatively affect their academic progress in the long run.

Noted that reliance by students on ChatGPT undermines cognitive development as well as critical thinking. If students do not assess the reliability as well as credibility of content generated through AI, they risk internalizing misinformation without questioning. Consequently, they lose their critical thinking as well as factchecking abilities which are a crucial part of learning at the tertiary level. (Li, 2024). The study emphasized that excessive utilization of AI applications such as ChatGPT can severely undermine students' personal learning capabilities. Creativity, communication, teamwork, as well as application of knowledge, which are all the key drivers of academic and professional success, can be weakened if students rely too much on computerized solutions instead of performing active learning processes themselves (Li, 2024). Overall, (Li, 2024) concluded that although AI technologies such as ChatGPT are valuable for helping with teaching and learning, it is important for students as well as teachers to understand their constraints. There must be a balanced and

responsible application of AI within learning contexts to avoid negative impacts on students' academic integrity, thinking patterns, and overall development.

Integrated AI Code Detection Systems

A lot of automated programming is now used for evaluation and security purposes in society, especially due to the growing popularity of LLM's or large language models (LLMs) such as ChatGPT. These models can increase productivity and help computers code more quickly, but have also introduced some problems to human-written code from AI-generated code. A study by Pan and (Chok, 2024) was presented to investigate the effectiveness of existing AI-generated content (AIGC) detectors in educational settings. The results showed that currently the existing detectors found it very difficult to accurately differentiate between human-written code and AI generated code, which would raise concerns about academic integrity. AI detects plagiarism and unusual coding patterns by leveraging sophisticated machine learning algorithms that analyze code structures, syntax. (Oedingen, 2024) developed models combining embedding features with supervised learning algorithms, achieving up to 98% accuracy in distinguishing human-written code from AI-generated code. (Suh, 2024) proposed approaches like finetuning large language models and using static code metrics, resulting in an F1 score of 82.55 in detecting AI generated code. The benefits of integrating AI into code detection are manifold. AI systems offer rapid analysis, handling vast amounts of code efficiently, which is invaluable in educational institutions and largescale software development. They also

enhance accuracy in identifying subtle patterns that might elude human reviewers.

However, these systems are not without limitations. (Pan and Chok, 2024) highlighted that existing AI generated content detectors often struggle to distinguish between human and AI generated code, especially when AI models are prompted to mimic human Coding Styles. This raises concerns about false positives, where genuine human written code might be misclassified.

Anti-Cheating Mechanisms in Coding Assessments

Cheating has continued to be a problem in education, and programmers are particularly subject to demanding and time-consuming computer coding assignments. Many studies have documented student misconduct and plagiarism in code coding exams. One study by (Chen, 2024) observed that students in an introductory computer science course submitted code using ChatGPT for developing and annotating AI annotations, submitted the code with comments relating to artificial intelligence processes, and later submitted code in response to those comments from subsequent review, this indicates a rethinking of students' cheating behavior to avoid detection (Chen, 2024). Another study, (Ferguson, 2023) investigated whether programming plagiarism can be detected using consensus-based analysis with a range of plagiarism detection tools such as MOSS and JPlag. One finding from the study showed that a significant portion of student submissions were indeed non-original (Ferguson, 2023).

Furthermore, (Zhou and Singh, 2023) show that AI generated code, especially code from large language models (GPT-3 or GPT-J) can run entire programming assignments without detection by MOSS tools which raise the possibility that existing academic integrity mechanisms are severely limited (Zhou & Singh, 2023). An additional finding by (Zhou, 2024) further indicates that students using ChatGPT in timed programming tests can program tasks significantly faster without achieving any noticeable improvement in computational quality which is an area of concern, in addition to concern about potential unfair advantage and less learning effort (Zhou, 2024).

This problem has worsened as students pay third parties to complete their programming assignments on their behalf. As observed by a case study reported by IEEE (Joy, 2002), this behavior is widespread in computer science education and not limited to the effective implementation of the current plagiarism prevention policies (Joy, 2022). In general, these findings highlight the need for effective plagiarism detection systems, clear academic honesty policies, and proactive strategies to prevent and actually experience learning of the first-order programming concepts.

In today's digital learning environment, maintaining integrity in coding assessments is becoming more complex. Traditional tools like MOSS and Dolos have long been used to detect code similarity, but recent studies suggest that they can be circumvented with minor code obfuscation. (Tian, 2020) introduced a tool called Mossad, which automatically modifies source code to fool traditional detectors, proving the limitations of these systems. To counteract such tactics.

(Huang, 2021) proposed BPlag, a behavior-based plagiarism detection tool that examines how code is executed rather than just its appearance. AI generated content; new challenges have emerged. (Strozanski, 2023) tackled this issue in his study Anti-Cheat for Programming Courses, where he developed ImGPT, a system that identifies whether a source code is written by a human or generated by AI. His work emphasizes the importance of adapting detection methods to keep up with AI capabilities. Behavioral monitoring is another layer of protection in online coding exams. (Tiong and Lee, 2024), in their study Advancing Online Assessment Integrity, combined IP address tracking with behavioral classification techniques such as tab switch detection and typing pattern analysis. Their integrated approach achieved a 90.7% accuracy rate in identifying academic misconduct.

These methods reflect how technical observation of student actions can enhance fairness in assessments. Expanding on this, (Zheng, 2024) proposed a comprehensive anti-cheating system that incorporates advanced technologies like head pose recognition, facial tracking, and pupil monitoring using computer vision models such as YOLOv5 and libraries like OpenCV and Dlib. Their system also includes security features like clipboard blocking and screenshot prevention using the Set Window Display -Affinity function in Windows, effectively deterring candidates from copying content or capturing exam material.

Several studies also explore whether these strategies genuinely discourage cheating. (Roberts and HaiJew, 2022) found that tools like webcam monitoring, keystroke logging, and blocking functions such as copy paste or

Alt-tab significantly reduce dishonest behavior. Even simple deterrents like honor pledges, randomized questions, or showing students, they're being monitored can have a psychological impact that promotes honesty.

Role of AI in Programming Evaluation and Security

Lately, Artificial Intelligence (AI) has been playing a bigger role in how we evaluate programming work and ensure code security. This shift has become especially important with the growing use of AI tools that can generate code. Traditionally, detecting code copying was all about finding similarities between students' submissions. The focus is changing new systems are being developed to figure out whether a piece of code was actually written by a person or generated by AI tools like ChatGPT or GitHub Copilot.

Introduced an approach that uses the perplexity of large language models (LLMs) to identify AI generated code. Perplexity, which measures how predictable a sequence of text is for a model, tends to be lower for AI generated content than for human–written code. Demirok and Kutlu (2024) proposed the AIG-CodeSet, an annotated dataset designed specifically to train and evaluate code detectors. (Xu and Sheng, 2024) The integration of AI in programming assessments offers several advantages. (AIRA Kumar, 2024), demonstrates how AI can provide instant feedback, detect bugs, and even recommend improvements. (Dunder and Lundborg, 2024) in comparing Kattis and ChatGPT in programming assessments, also highlighted how AI can assist in real-time evaluation, flagging potential anomalies in coding patterns that might escape manual review.

AI driven evaluation is not without its limitations. One significant concern is the possibility of false positives cases where human–written code is mistakenly identified as AI generated. (Bukhari, 2024) emphasized that detection models can be sensitive to certain programming styles of templates which may unfairly affect

students who follow standard coding patterns. (Arias, 2024) also pointed out that while tools like ChatGPT are powerful, their role in educational settings must be critically assessed, as over–reliance can diminish learners' problem–solving development.

AI Code Detection

Artificial Intelligence (AI) has influenced the field of computer science, especially on how programming tasks are approached today. With the rise of large language models (LLMs) like ChatGPT, Gemini, and Claude, it has become possible for developers to automatically generate high-quality code snippets, debug functions, or even build full applications with minimal manual intervention. While these developments enhance productivity and accessibility for learners and professionals alike, they also pose a challenge in educational settings where originality and skill development are expected (Pan, 2024). The ability to distinguish between AI generated code and student authored code has become more important than ever. Instructors must ensure that students genuinely learn programming skills rather than relying solely on AI assistance. Moreover, the automated generation of code can introduce undetected vulnerabilities, bad practices, and bugs that students may not fully understand. If not properly

regulated or detected, it could undermine the very purpose of programming education, which is to teach logical thinking, problem-solving, and technical creativity (Oedingen, 2024). To address these concerns, AI code detection tools have been developed. These tools analyze source code submissions and assess whether they exhibit characteristics typical of machine generated outputs. They consider factors such as syntax uniformity, pattern predictability, and linguistic features specific to AI driven code. By implementing these technologies, educational institutions can uphold academic integrity, maintain high teaching standards, and better prepare students for real-world software development careers.

CodeSpy

Codespy is a specialized detection tool built specifically in order to identify AI generated code in popular programming languages like Java, Python, JavaScript, C++, C#, and PHP. Unlike other general-purpose AI detectors, Codespy focuses entirely on programming languages, making it highly accurate for source code analysis. According to its developers, Codespy achieves an impressive 98% detection accuracy rate by analyzing deep syntactic structures and semantic patterns unique to human versus AI written code (CodeSpy, 2024).

One of Codespy's biggest advantages is its commitment to educational accessibility. It offers a **free plan** that allows students and teachers to scan code

submissions without any payment, a rare feature among detection platforms. On top of that, it offers smooth integration into Visual Studio Code, allowing real-time

checking while students are actively writing their code removing the need for manual uploads to an external website. This level of convenience not only saves time but also encourages proper programming practices by providing immediate feedback (CodeSpy, 2024).

Besides detection, Codespy also highlights potential security vulnerabilities in AI generated code. This is extremely important for computer science programs because it helps students not only focus on authenticity but also develop an eye for safe coding practices. When compared to more generic detectors, Codespy clearly tailors its functionality towards the needs of both the academic and professional development communities.

GPTZero

GPTZero originally made headlines as one of the first detectors aimed at spotting AI generated text from language models like ChatGPT. Its method relies on linguistic markers such as "perplexity" (how unpredictable a text is) and "burstiness" (how varied sentence structures are) to predict if something was AI written (Pan, 2024). While this technique is fairly effective for essays or articles, it struggles significantly when applied to programming code, where the structure is inherently rigid and repetitive.

In terms of accessibility, GPTZero offers a free version that allows basic detection, making it relatively easy for users to start testing documents immediately. It has gained popularity particularly among academic institutions for essay detection. However, its interface and functionality are mostly geared

towards natural language processing rather than code syntax analysis, meaning programming instructors would not find it reliable for student submissions in computer science courses. The main drawback of GPTZero for NEUDev is that it simply was not built for code. When analyzing structured programming languages, GPTZero often either produces false negatives or fails to recognize subtle AI generated patterns. Its inability to accurately distinguish between a logically human coded script and an AI crafted one makes it unsuitable for our intended application within NEUDev.

Codeleaks

Codeleaks positions itself as a reliable and strong AI detection tool that handles over 100 programming languages and specializes in source code plagiarism detection. It is often used by businesses and software development institutions to guarantee compliance and protect intellectual property. The platform is effective when it comes to identifying AIgenerated code submissions, making it a solid contender in the AI detection market (Codeleaks, 2024).

However, Codeleaks has several limitations, particularly in terms of transparency and accessibility. First, there is very little publicly available information about their subscription plans. The lack of clarity around pricing and features makes it difficult for students or universities operating under tight budgets to adopt the platform with confidence. Secondly, it does not seem to offer direct IDE integration, meaning users have to manually upload files through its web portal, adding friction to the detection process. Without clear pricing models and with missing usability features such as real-time detection inside

coding environments, it cannot match Codespy's seamless educational experience.

ZeroGPT

ZeroGPT is another detection tool that claims to have a high success rate for identifying AI written content. Like GPTZero, it primarily targets natural language rather than code. It provides a generous free plan that allows up to 15,000 characters per analysis and supports multiple languages, making it attractive at first glance (Originality.AI, 2024).

Despite its accessibility, ZeroGPT's model isn't well-suited for programming code. Language detection algorithms that evaluate "writing style" metrics perform poorly when evaluating structured syntax like C#, Python, or Java. In fact, programmers often follow highly standardized syntax conventions, leading to frequent misclassifications when evaluated through models optimized for text randomness. Therefore, ZeroGPT lacks the technical depth needed for accurately detecting AI generated source code in academic coding environments.

BlueOptima

BlueOptima's Code Author Detector is a tool made mainly for large enterprises to figure out if a piece of code was written by a human or generated by AI. It works by analyzing developer behavior, code structure, and stylistic patterns across a project. It can also integrate with Git, Jira, and Bitbucket, so it becomes part of the normal software development workflow (BlueOptima, 2024).

A good thing about BlueOptima is that it has strong detection capabilities at a large scale. It can monitor developer activities across hundreds of repositories and highlight suspicious AI usage. However, it is expensive. Pricing usually starts at a custom enterprise quote, which is often thousands of dollars per year, depending on the size of the team (BlueOptima, 2024).

Code Vision

CodeVision is a new AI code detection method that treats source code like an image. It turns the code into a 2D map based on token probabilities and then uses vision models like ResNet and Vision Transformers (ViT) to analyze the spatial structure of the code (Li, 2024). This method captures indentation, bracket positioning, and patterns that AI models often leave behind. One advantage of CodeVision is its accuracy, especially in experiments. It can detect AI generated code even across different programming languages like Python, Java, and C++. However, it is still mainly a research project. There's no free tool, paid service, or easy access for schools or developers to use it right now. You would have to manually reproduce their research setup to even try it (Li, 2024).

GPTSniffer

GPTSniffer is an AI code detector that works by using a model called CodeBERT. It looks deeper into the structure and meaning of the code, not just at surface text patterns like other basic detectors (Wang, 2024). Studies showed that GPTSniffer performed way better than GPTZero and OpenAI's old classifiers when it came to detecting AI written code.GPTSniffer was made specifically for source code, not essays or chat conversations. This means it understands code

logic and structure better. However, just like CodeVision, it's only available as an opensource academic project. There's no hosted platform where students or teachers can easily upload code and detect AI usage. You would need to have machine learning expertise and your own server to even use it properly (Wang, 2024).

Table 2.1 Comparison of AI Code Detection

Tool	Benefit/s	Drawback/s	Cross Benefit
CodeSpy	• Free for students and teachers • 98% detection accuracy • Realtime VS Code integration • Detects AI code and security risks	• Limited support for rare languages	Best option. High performance with zero cost. Perfect for NEUDev.
GPTZero	• Free version available • Easy to use for text essays	• Not reliable for code detection • No coding tool integration	Good for essays, but low practical use for coding activities.
Codeleaks	• Supports 100+ languages • Good AI code detection theoretically	• No clear pricing • No clear setup instructions	Promising, but not transparent enough for NEUDev use.
ZeroGPT	• Free limited plan • Easy online access	• Built for text detection, not code • Inaccurate for structured syntax	Accessible but unreliable for programming assessments.
BlueOptima	• Very powerful at enterprise scale • Deep code analysis	• Very expensive • Requires complex deployment	Technically strong, but financially and logistically impractical for schools.
CodeVision	• High experimental accuracy • Multilanguage support	• Still in research • No public platform or pricing	Promising academically but unusable for real deployments.
GPTSniffer	• Very accurate for detecting AI code • Finetuned for programming languages	• Needs selfhosting and ML expertise • No public hosted tool	Technically strong, but very hard to implement for NEUDev.

Table 2.1 presents a comparison of seven (7) AI-based tools designed to detect AI- generated content, with a particular focus on their usefulness in identifying AI-generated Code. Each tool is evaluated based on its benefits, drawbacks, and potential value in academic or educational settings. Especially for programming courses.

CodeSpy stands out as the most suitable option for educational environments. It offers several advantages, including being free for both students and teachers, high detection accuracy (98%), integration with Visual Studio Code, and the ability to detect both AI-generated and insecure code. However, it has limited support for less common programming languages. Despite this minor drawback, it is considered the best overall option due to its strong performance and accessibility, particularly for academic use.

GPTZero, while offering a free version and being user-friendly, is more effective for detecting AI-generated text in essays rather than code. It lacks coding tool integration and is not reliable for code detection, making it more appropriate for writing-based assessments than programming evaluations.

CodeSleuth shows promise due to its support for over 100 programming languages and its theoretical ability to detect AI-generated code. However, its lack of transparent pricing and unclear setup instructions limit its practical use in academic settings. Without clear guidance or access, its adoption in schools remains uncertain.

ZeroGPT provides easy online access and a free limited plan but is primarily designed to detect AI in natural language text rather than code. It struggles with structured syntax, making it unreliable for code-related assessments. Therefore, while accessible, it is not recommended for use in programming courses.

BlueOptima is a powerful tool capable of deep code analysis and is widely

used at the enterprise level. However, it is very expensive and requires a complex deployment process, which makes it impractical for most academic institutions. Its technical strengths are outweighed by its financial and logistical limitations in a school context.

CodeVision demonstrates high experimental accuracy and supports multiple programming languages. However, as it is still in the research phase and lacks a public platform or pricing model, it is not yet a viable option for actual deployment in academic environments.

Finally, **GPTSniffer** appears to be highly accurate and specifically designed for detecting AI-generated programming code. Nonetheless, it requires significant setup and machine learning expertise, and it does not currently offer a public tool. As such, while technically promising, it would be difficult to implement in a typical educational setting. In summary, although several tools offer unique advantages, most are either too specialized, costly, or underdeveloped for widespread academic use. **CodeSpy** is the most practical and effective option for detecting AI-generated code in educational environments due to its high accuracy, ease of integration, and zero cost.

AI Coding Assistant

AI Coding Assistants are becoming valuable tools for both students and developers by helping them write better code more efficiently. These assistants,

like GitHub Copilot, use artificial intelligence to suggest code, fix errors, and even explain programming concepts in real-time. (Millam and Lindstaedt, 2024), they found that using AI tools in programming education made learning easier and more engaging, especially for beginners. Instead of just giving the answer, these assistants guide students through the process–making it feel like they have a personal tutor while still encouraging independent thinking. As more coding assessment platforms adopt AI features, these assistants are helping create a more supportive and interactive learning environment that not only boosts productivity but also builds confidence in coding.

AI ChatBots

Technology has recently been more prevalent than ever; it is evident and can be observed in various aspects of daily life. Recently, the public has been 'humanizing' AI as it simulates the cognitive abilities of a human. Gupta, Hathwar, and Vijayakumar, these chatbots are built using natural language processing and machine learning, which allows them to understand questions and respond in a way that feels interactive. In learning environments, especially coding platforms, chatbots can be used to guide students, provide instant feedback. They make learning feel easier by giving answers right when students need them. Instead of just being simple bots that reply, these chatbots act more like helpful study buddies guiding students through their lessons in a way that feels personal, supportive, and efficient.

AI Test Cases Generation

AI powered text case generation is changing the way software testing is done. Traditionally, writing text cases is a time-consuming task that requires

developers to manually think through all the possible ways code might break. With AI, especially through generative models, this process can now be automated. (Baqar and Khanda, 2024), AI systems can learn from existing code and testing patterns to automatically create meaningful test cases that help catch bugs early. This not only speeds up the testing process but also improves coverage by exploring edge cases developers might overlook. (Tendulkar, 2024) highlights how generative AI tools are being used not only for test creation but also for defect detection ensuring software quality with minimal human intervention. These tools are capable of understanding the logic of the application, predicting potential failure points, and generating tests accordingly. In the context of coding assessment platforms, AI generated text cases can be used to validate students' solutions more effectively.

AI AutoGrading

AI powered autograding systems have become a vital part of modern coding assessment platforms, addressing longstanding challenges in programming education such as grading consistency. (Vetrivel, 2023), highlighted that the use of automated grading systems not only helps lessen the workload of instructors but also supports prompt and objective evaluation of programming tasks. The authors noted that these systems are particularly advantageous in large class settings, where manual grading can be inefficient and prone to errors. (Gambo, 2024), presented GRADAI an automated grading system that combines

artificial intelligence with human supervision to promote both grading accuracy and educational value. The system not only checks for correctness but also analyzes code quality and style, enabling a more comprehensive evaluation. Both studies support the implementation of AI in platforms like NEUDev, which aims to automate code assessment, enhance feedback delivery, and uphold academic integrity through intelligent evaluation mechanisms.

Synthesis

In this part of the paper, the synthesis presents an integrated overview of the gathered literature, offering insights into recurring themes, key findings, and gaps identified across the selected sources. This synthesis aims to contextualize the relevance of existing studies to the current research, highlighting how these works collectively inform and support the objectives of the study.

Programming Environment Supporting Python, Java, and C#

Compared to other languages, Python, Java, and C# are important in programming education as they are all applicable in different paradigms and are popular in both academia and industry. Apart from these, over the recent years, Python has become a popular teaching language because of its simple syntax and flexibility, in schools and universities introductory courses are now switched from Java to Python because of its "*simplicity, flexibility, and [extensive] libraries*" that make it ideal for learning as well as for applying it in data analytics applications (Guo, 2021). Python's popularity is reflected by its status as one of the most widely used languages today; it was ranked the world's most popular coding language by IEEE and dubbed "Language of the Year" in 2018 (Jansen,

2024). In truth, Python has become the usual first language to be used by new programmers because it is often thought of as the "*easiest to learn*" general purpose language due to the huge demand for software skills (Krill, 2024).

Meanwhile, Java is still a core learning tool in computer science and software development. For years it has been the go-to tool to teach object-oriented programming and largescale application design. Even today, the use of Python is declining, but still, Java is "among the most popular languages" in industry rankings (Krill, 2024). A lot of high school and university courses still use Java to demonstrate strong typing skills and basic concepts and it consistently ranks in the top handful of languages used professionally. C# On the other hand is important for students to learn because of its presence in the industry as a strong programming language that has been consistently placing in either top 10 or top 5 ranking worldwide, in both the. NET ecosystem and for developing games (Unity), and in enterprise applications (Krill, 2024). And, as noted earlier, C# is both a very similar language in concept, it is an object-oriented (statically typed) language and

comes from Microsoft, but also exposes students to the Microsoft stack which are skills a lot employers value.

Modern curricula or platforms will often focus on one language (like, Java or Python), which limits students' exposure to different programming paradigms and actual tools used in the real-world scenarios. Thus, resulting in lack of preparation for jobs that require fluency in more than one language. Learning

Python, Java, and C# together in one platform can help students learn basic programming concepts through easy to learn language (Python) while practicing strong software engineering principles (especially those used in modern industry, such as Java and C#) (Islam, 2024).

Detecting AI-Generated Code (CodeSpy)

In recent years, with the development of AI assisted coding, (i.e., student code generation using ChatGPT or similar AI models to generate assignment solutions), substantial problem arise in terms of academic integrity: code that is original, and not copied (not from peer reviews or online sources) and not truly created by the student. Traditional plagiarism scanners lack in this respect. The idea is that they must rely on mutual similarities between submissions, or known code, and because of it, "t*heir detection algorithms cannot detect LLM code generated; in fact, LLM code is generated in a stochastic procedure, or it does not necessarily correspond to any existing source*" (Winter, 2024). In other words, one might ask an AI to write some code. The solution produced by the machine would be completely unique. This problem is especially serious when we consider current academic integrity measures for programming courses. CodeSpy solves this problem using machine learning to identify patterns in AI written code and highlight suspected lines for teachers to act upon (CodeSpy, 2024). A recent analysis showed that CodeSpy has a claimed 98% accuracy rate when checking AI generated code written in languages like Python, Java and C#, and a low-cost first tier pricing model, thus making CodeSpy a better choice for AI code detection in education (CodeSpy, 2024). Most existing tools cannot

detect AI generated code. By using CodeSpy, a tool to detect submissions with high AI likelihood, teaching staff can effectively maintain academic integrity (Leinonen, 2024; CodeSpy, 2024).

Similarity Checking for Code (DOLOS)

Detection of plagiarism (unauthorized collaboration) in coding assignments has been long seen as a concern in education. Recent studies have indicated that students copy entire sections of codes. However, most teachers do not regularly use plagiarism detection tools due to the lack of practicality or interpretability (Maertens, 2022). Academic dishonesty may be unchecked or caught only by manual analysis. Turnaround code plagiarism detection tools have been available for many years and are frequently used but generally only provide textual/token similarities and require the instructor to read through the raw results in order to confirm plagiarism (Winter, 2024). Older tools are not always able to compensate for things like refactoring and obfuscation, students might create variable names themselves or reformat code in order to hide a copy. Instead, what teachers need is an easier, more reliable plagiarism detection tool which not only finds matches, but also helps visualize and prove them.

Recent research in DOLOS has set this up as a next generation plagiarism detection system for source code. Rather than using language specific tools, DOLOS is "*language agnostic"* in that it supports various programming languages (Maertens, 2022). This is crucial in a teaching platform as a single system can deal with consistent reporting for all assessments. DOLOS uses state-of-the-art plagiarism detection algorithms (for example, efficient parse tree

or token sequence matching with normalization), and pairs them with interactive visualization (Maertens, 2022). The latest version of DOLOS provides a web-based interface with dashboards and graph views that show clusters of similar code submissions. So, an instructor can quickly see which assignments or assessments are suspiciously similar to each other, and then drill down to see the code in their groups. Visual evidence like these is extremely valuable in academic integrity proceedings. It gives an instant assessment of whether a collection of source files contains suspected cases of plagiarism and how widespread it is through intuitive graphs and color-coded similarity metrics (Maertens, 2022).

Automated Code Evaluation

Automated grading of code (auto-grading) has become an increasingly important part of programming education, particularly in larger classes. A number of studies and literature reviews have examined the effectiveness of automated grading systems, and they generally agree that there are key advantages over just grading the code by hand. According to the authors: Automated programming evaluation systems apply the same criteria to all submissions, thereby eliminating human biases and random variance (Sokač, 2025). By applying predefined test cases and metrics, an auto-grader allows each student's code to be rated against the same standard (each student is rated against the same standard in a consistent manner). These characteristics make them highly fair. In fact, automatic evaluation systems can "correct errors made by human graders" by adherently following the same rubric (Sokač, 2025). For example, a

human might not include all the required parameters, or credit partially incorrectly, but an effective auto-grader will do so uniformly. Their consistency is especially important in large courses, where perhaps dozens of teaching assistants may be grading with slightly different leniency than other students' code. Experimental research into human grading has found that people's assessments of the same partially correct solutions vary significantly. Also, manual grading can be error-prone and slow (Sokač, 2025). Automated evaluation eliminates this variability: it provides efficient and objective grading of a large number of student solutions with a uniform standard (Sokač, 2025).

Automated code evaluation can also provide results in seconds or minutes, as opposed to the days or weeks it would take a human to grade a batch of large assignments. This short turnaround is an advantage of iterative feedback. Studies have shown that most automated tools are capable of fully automated assessment with almost instantaneous feedback and multi-resubmission (Messer, 2023). This has a positive effect on students: they receive timely feedback about mistakes and can attempt to fix their code and resubmit (often within the same assignment deadline).

This early feedback loop has shown to have a positive impact on student engagement, satisfaction, and success, as learners have "more opportunities to succeed" and are able to learn from their mistakes in real-time (Messer, 2023). In an instructional sense, automation means an instructor can effectively teach a class of hundreds (or an online MOOC for thousands) without having an unmanageable grading workload (Jukiewicz, 2023). One paper explicitly states:

“automatic code checking allows teaching and grading large groups of students without increasing the workload for teachers,” showing the scalability advantage in higher education (Jukiewicz, 2023). Students who used auto-graders in this area found it particularly important during periods where one-on-one instruction was out of reach, with very limited resources for traditional one-on-one help, auto-graders allowed large classes of students to continue and provide rapid evaluation and feedback to students.

Reliability of AI Based Grading Evaluation

With advancements in AI technology, there has been a rapid rise in interest in using AI for programming assignments in a wider context. This could mean using machine learning models to predict a score; AI systems to provide feedback; or even human style or solution orientation assessment. A concern then arises: Determining whether AI generated grades for submitted code can be considered reliable and valid in academic settings. Recent research shows that, with appropriate design and scrutiny, AI-based grading can indeed accurately approximate human graders and even offer some improvements. For instance, (Sokač, 2025) proposed an AI grading framework using contrastive learning for SQL programming tasks.

The results indicate that the approach “improves the accuracy, consistency, and transparency of automated grading, aligning more closely with human grading standards.” (Sokač, 2025). In their case, the AI model could compare a student’s SQL solution against a correct solution in a high dimensional feature space and give a score (percentage of points deducted) that

corresponds to what a human grader would score. To further optimize the system, the researchers proposed explainable AI techniques (integrated gradients and token-level attributions) so that AI decision-making could be understood by the students; these tools allow for the interpretability of which parts of the student's code were wrong, and how that affected their grade (Sokač, 2025). This resulted in an AI grading tool that provides "clear, actionable feedback" whereas matching consistency across all categories, just as how a human would grade.

AI based grading systems can greatly reduce the human workload for large classes without necessarily sacrificing the quality of evaluation. AI can consider complex criteria, not only binary test results, however, an AI grader can be trained to determine whether a specific style of code is good, whether the solution is efficient, or whether the student has chosen good variable names. Moreover, since the AI can also be trained on examples of human grading, it can capture expert knowledge and apply it consistently. Studies often validate such systems by comparing AI assigned grades to human assigned grades. In fact, according to a literature review, most papers evaluating automated assessment tools "compare the automatic assessment tools to grades or feedback provided by human graders. (Messer, 2023). When the AI and human grades are highly correlated, it increases confidence that the AI system is reliable. Also, AI can sometimes catch problems humans might miss, for instance, subtle logical errors or suspicious patterns, especially with the plagiarism detection and testing tools described earlier. AI grading is fast: what might take a human several minutes (or

more) per program, especially if they're giving written feedback, it can do in seconds. This speed can lead to more frequent assessment and feedback cycles that are beneficial for learning. It also solves the problem of grading consistency over time; humans might get tired or might unconsciously grade differently at the beginning versus the end of a long stack of assignments, while an AI's criteria don't drift over time. Research in education technology emphasizes that accuracy, consistency, and quality feedback are three key goals of automated programming assessment systems, and AI can help optimize all three (Soka, 2025). By providing clear grading criteria in an AI system, we can even "correct [some] errors made by human graders" and ensure that no student is unfairly advantaged or disadvantaged by who grades their work (Soka, 2025).

CHAPTER 3

METHODOLOGY

This chapter presents the development process of the NEUDev platform. It covers the system requirements, architecture, and key components including the frontend, backend, code execution engine, and databases. It also discusses the integration of API tools, project evaluation methods, and the testing dataset used to validate system performance.

Project Design

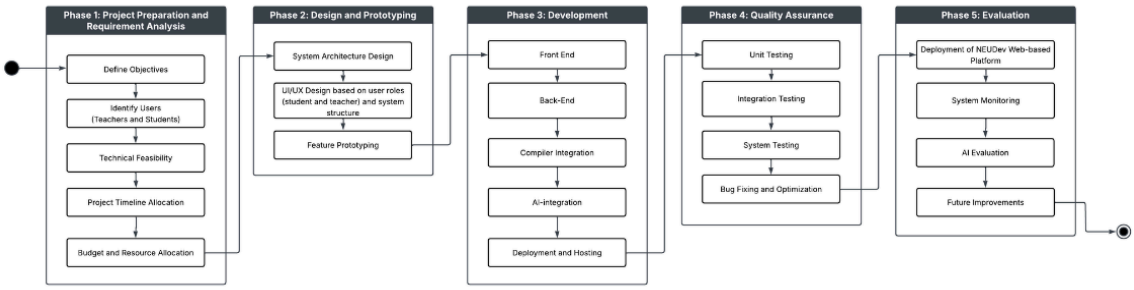


Figure 3.1 Project Design

The project design provides the projects different phases such as preparation, design and prototyping, the development, testing, and evaluation. This section has the overview of all the phase of the NEUDev's process and implementation.

Phase 1: Project Preparation and Requirement Analysis

The first phase of the project is the preparation and analysis of the goal and objectives. In this phase, the analysis of the gaps and problems we're addressed. This involves the identification of the purpose of the project, the one who will use the systems, the key outcomes, and the goal the project needs to

achieve. During this phase, research and analysis of existing studies are conducted to find gaps, inefficiencies, and challenges in different coding assessment platforms and programming assignment checking. Creation of requirements and plans is a big part of this phase. In this phase, the project outlines, plans, designations, works, roles, and responsibilities are discussed. It also includes the identification of the stakeholders, finding tech stacks that ensures the feasibility of the project, creation of timeline and Gantt charts are presents and budget and allocations are also discussed.

Phase 2: Design and Prototyping

The second phase of the project focuses on designing the system architecture and visualizing how the platform would work. t starts with System Architecture Design, where the entire system structure, including its components, data flow, and module to module interaction, is outlined. This gives a clear technical development blueprint. This is followed by the UI/UX Design, which is developed as per the requirement of various user roles, for example, teachers and students. In this stage, the interface is created with a focus to provide usability, accessibility, and seamless user experience, while corresponding to the system's structure and functionality. After the initial design is established, Feature Prototyping occurs. This is the process of creating a functional model or simulation of key features to demonstrate the key functionalities of the platform.

Phase 3: Development

The third phase is about the mass development of the NEUDev platform. It starts with FrontEnd Development using ReactJS to develop the student and

teacher interface from accepted UI/UX designs. This is followed by BackEnd Development using Laravel to handle data processing, API endpoints, user management, and real-time activity tracking. There is also the Compiler Integration, which provides in browser code execution through WebSocket, with support for programming languages such as Python, Java, and C#. The AI Integration provides sophisticated features such as code similarity detection (Dolos), AI generated code detection (CodeSpy), and rubric based scoring. Finally, the system goes through Deployment and Hosting in environments such as Railway, where databases are created and the application is exposed and secured to be accessed in the real world.

Phase 4: Quality Assurance

The fourth phase ensures the reliability and stability of the NEUDev platform by having a very rigorous testing process. It begins with Unit Testing, in which the separate functionalities and components are tested independently to ensure correctness. Integration Testing is then performed to test the combined functionality of different modules. Then System Testing is performed to validate the platform as a whole, confirming that all its features behave as such under various user scenarios. Finally, Bug Fixing and Optimization are performed to fix any bugs discovered, improve performance, and prepare the platform for rollout and real use.

Phase 5: Evaluation

The fifth phase is focused on measuring the operational effectiveness of the NEUDev web platform implemented. It starts with the Platform Deployment,

and then there is System Monitoring ongoing, to offer stability and functionality in real use. In this, Bug Tracking and Fixes are performed to address any issues that have arisen. An AI Evaluation is also performed to ascertain the efficacy and correctness of the implemented AI features such as code detection and automated marking. The performance attained in this phase is then used to produce Future Improvements, thus enabling the system to be developed according to real user feedback and performance measure.

Conceptual Framework

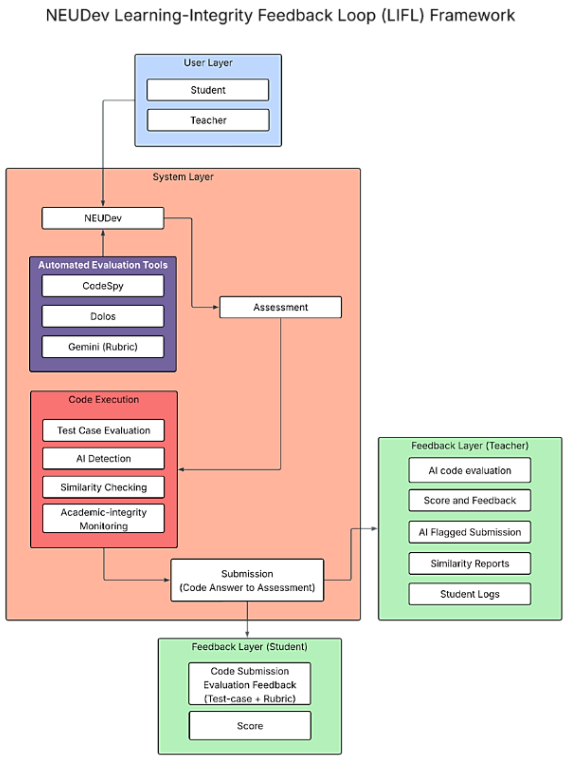


Figure 3.2 NEUDev Learning-Integrity Feeback Loop (LIFL) Framework

The Learning Integrity Feedback Loop (LIFL) Framework serves as a custom conceptual model that demonstrates the core logic of the NEUDev coding assessment platform system. The framework is built around 3 layers: User, System, and Feedback. This showcases how students and teachers

interact with the system, how the system processes and evaluates code submissions with the utilization of AI tools, and how the feedback layer operates to promote academic integrity. This approach allows the researchers to demonstrate how the system features are utilized to promote student learning integrity while also considering moderating factors such as a student's programming skill level.

The platform involves two primary user types: ***Students*** and ***Teachers***. Students participate in assessments and receive feedback based on their submission and performance. Meanwhile teachers create assessments and review the system generated reports.

The ***System Layer*** serves as the core operational component of the NEUDev's system, it comprises two subsystems: Automated Evaluation tools (CodeSpy, Dolos, and Gemini [Rubric]) and Code Execution Subsystem. ***CodeSpy*** detects and flags if the code submitted by a student is AI generated, ***Dolos*** performs code similarity analysis to detect potential plagiarism amongst students, and finally, ***Gemini (Rubric)*** provides assistance in code evaluation feedback based on problem solving quality, logic, and code standards.

The Code Execution Subsystem executes and evaluates the student's submitted code using several processes: Testcase evaluation checks the code's correctness against predefined outputs, AI Detection utilizes CodeSpy to flag AI generated submissions, Similarity Checking utilizes Dolos to compare submitted codes and detect potential plagiarism or cheating, and Academic Integrity

Monitoring tracks tab switching, disables copy paste attempts, and other academic behavior violations.

The ***assessment*** modules represent the actual assessments deployed and disseminated by teachers. These are the actual programming tasks administered to students. Once an activity is initiated, the System Layer starts its work behind the scenes. After a student submits their code corresponding to the assessments, it is processed by the automated system. The student code submissions serve as the central datapoint for the system to analyze, evaluate, and generate feedback and scores.

The ***Feedback Layer*** then provides feedback for both students and teachers. The student is provided with post submission feedback, detailed with the score rubric as to why they received their particular score, it includes a testcase based, rubric based, and overall code quality evaluation. This supports a student's education through iterative reflection and improvement. Meanwhile, teachers have access to different tools assisting them in monitoring evaluative outputs such as AI flagged/detected submissions, Similarity checks, Evaluation summaries, and Student's behavior logs.

This framework demonstrates how the NEUDev web based online platform operates as feedback oriented and AI enhanced educational tool. It has a balanced assessment automation with academic integrity checks and helps both students by providing feedback and teachers by having assistance with monitoring student behavior.

Frontend Layer

The frontend of NEUDev system is developed using ReactJS, styled with Bootstrap 5 for responsive design and custom CSS for theming. The project is bundled and served using Vite, a fast and modern frontend build tool optimized for development and production environments.

Table 3.1 Frontend Technologies Used

Tool/Library	Purpose
ReactJS	Frontend framework for creating UI components
Vite	Fast build and development server
React Bootstrap	Prebuilt UI components integrated with Bootstrap
Xterm.js	Terminal emulation for code output
Monaco Editor	In browser code editor (similar to VS Code)
Font Awesome	Iconography support
React Router DOM	Routing and navigation management
AOS (Animate on Scroll)	Scroll based animation effects
JSZip & FileSaver	File compression and download functionalities

To effectively develop a responsive, interactive, and user-friendly interface for NEUDev, different front-end tools were utilized. Each tool was chosen based on its relevance to the platform's requirements. Below is a detailed explanation of each tool:

ReactJS. ReactJS is a JavaScript library used to develop a dynamic and re-usable user-interface. It allows the development of single-page application (SPA) so that the content updates dynamically without requiring full page reloads.

Vite. A modern front-end tool that provides a fast development environment It offers instant server start, and efficient bundling whoch improves

the overall development workflow and performance of the frontend.

React Bootstrap. Integrates the Bootstrap's responsive design with the React components. It simplifies the development of the user-interface using pre-styled components such as modals, buttons, and forms, ensuring design consistency and faster UI prototyping.

XTerm.js. A JavaScript library that provides a terminal emulator for the web. In the context of NEUDev, it is used to simulate a console output for code execution, which allows students to see the results of their code in real time, similar to a traditional coding environment.

Monaco Editor. The same editor used in Visual Studio Code, brought to the browser. It includes syntax highlighting, autocompletion, and error checking. This is important for providing students with an intuitive and familiar coding interface during assessments.

Font Awesome. A wide set of vector icons that can be easily customized to enhance the visual design and usability of the UI, particularly for buttons, menus, and status indicators.

React Router DOM. Manages navigation and routing within the application.

AOS (Animation on Scroll). A library that enables animations as users scroll through the page adding interactive and visually appealing effects.

JSZip and FileSaver. Libraries that are used for file compression and download functionality. Enables the creation of ZIP files directly in the browser, while FileSaver allows for saving files on the client side. Combined together, they support features like exporting code submissions or downloading activity data.

Backend Layer

The backend of the NEUDev system is implemented using Laravel 11, a modern PHP web framework. Laravel follows the Model-View-Controller (MVC) design pattern and is widely used for building RESTful API applications. Laravel handles all business logic, database operations, API routing, authentication, and communication with external services like Docker containers, CodeSpy, and DOLOS.

Table 3.2 Backend Technologies Used

Tool/Library	Purpose
Laravel Framework (v11)	Main backend framework; MV-Cbased structure
PHP (v8.2)	Primary backend programming language
Laravel Sanctum	API authentication (token-based) for SPA security
Eloquent ORM	Laravel's built-in ORM for database interactions
Laravel Sail	Dockerized local development environment
Pest PHP	Laravel native testing framework
Laravel Pail	Realtime log monitoring and terminal debugging
Laravel HTTP Client (Guzzle)	Used for making outgoing HTTP requests to external services (e.g., Dolos API for similarity checking)
REST API Architecture	Enables structured communication between the Laravel backend and React frontend
Laravel Events / Broadcasting	Allows backend to trigger systemwide events like deadline updates (e.g., Deadline Changed)
Laravel Mail System	Sends transactional emails using `Mailable` classes (e.g., teacher verification notifications)

To effectively power the features of NEUDev from the server side, several backend tools and technologies were used. Each of these tools was selected based on how well it fits the needs of the platform, such as handling code submissions, database operations, real-time updates, and system security. Below is a detailed explanation of each backend technology:

Laravel Framework (v11). The main backend framework used for NEUDev. It follows the MVC (Model-View-Controller) structure, which helps organize the code in a clean and manageable way, making development easier and more efficient.

PHP (v8.2). The main programming language used behind Laravel. It runs the logic that powers the backend, like saving data, handling user requests, and managing assessments.

Laravel Sanctum. Used to secure our system, especially for single-page applications (SPAs) like NEUDev. It provides token-based API authentication, making sure only the right users can access protected parts of the system.

Eloquent ORM. Eloquent is Laravel's built-in Object-Relational Mapper (ORM). It allows us to work with the database using PHP instead of raw SQL. This makes database operations like inserting or updating records much simpler and more readable.

Laravel Sail. A tool that helps set up the local development environment using Docker. It allows developers to run the app on their own machines easily without manually installing all the required software.

Pest PHP. A testing framework that is built specifically for Laravel. It's used to test if all backend features (like API endpoints and logic) are working properly

Laravel Pail. This tool provides a real-time view of backend logs and helps

in debugging. It's useful for tracking errors and understanding what's happening in

It's useful for tracking errors and understanding what's happening in the system during development or testing.

Laravel HTTP Client (Guzzle). Used for sending HTTP requests to external services. For example, NEUDev uses it to connect to the Dolos API for checking code similarity in student submissions.

REST API Architecture. REST (Representational State Transfer) is a common architecture used for communication between the frontend (ReactJS) and the backend (Laravel). It ensures data is sent and received in a structured and predictable way.

Laravel Events / Broadcasting. Allows the backend to trigger system-wide events. For instance, when a teacher updates a deadline, the system can automatically notify all affected users in real-time.

Laravel Mail System. Used to send emails, like verification messages to teachers. It uses special mailable classes that make email creation and sending easy and organized.

Code Execution Engine

The NEUDev Code Execution Engine is a custom-built, Web Socket powered, Docker supported runtime service designed to compile and execute student submitted code. It is based on a Codex API, modified to support multiple programming languages (Java, Python, and C#) and provide a real-time terminal.

Table 3.3 Code Engine Integration

Component	Purpose
NEUDev Compiler (Codex API)	Used for hard execution in real containers with real compilers
Laravel Backend	Stores output, and returns results
React Frontend	Displays output

To allow students to run and test their code during assessments, NEUDev uses and integrates a code execution engine. This involves several components working together to take the student code, run it securely, and return the results in real time. Below is a detailed explanation of how the code engine integration works:

NEUDev Compiler (Codex API). This is the main component responsible for running the code. It uses real containers with real compilers to execute the

student's code. This is called a "hard execution," meaning the code is actually compiled and run in a safe environment and not just simulated.

Laravel Backend. Once the code is sent for execution, the Laravel backend handles the response. It stores the output (for example, the result of the code or any error messages) and then returns it to the frontend so it can be displayed to the user.

React Frontend. After the backend processes the code output, the React frontend is in charge of displaying the result to the student. Whether the code runs successfully or throws an error, the frontend presents the output clearly in the coding interface.

Database

The NEUDev system utilizes MySQL, an opensource, relational database management system (RDBMS), to store and manage all persistent data. MySQL serves as the central data store for all users, submissions, classes, etc.

API Tools Integrated

CodeSpy AI Code Detection

CodeSpy is a pretrained machine learning API designed to detect AI-generated source code. It evaluates code submissions and returns a score indicating the likelihood that code was written by an AI, helping uphold academic integrity in programming assessments. NEUDev uses CodeSpy to flag potential misuse of AI generated solutions.

Dolos Similarity Code Detection

Dolos is a code similarity detection tool developed for educational environments. It analyzes source code across multiple student submissions to identify instances of code reuse or plagiarism. NEUDev integrates Dolos to automatically compare submitted solutions and generate similarity scores and match reports.

Gemini API Rubrics

The Gemini API, a free large language model (LLM) provided by Google AI Studio is integrated into NEUDev to assist in evaluating the structure and quality of student code submissions. Unlike traditional compilers, Gemini not only interprets and simulates program outputs but also assesses the underlying logic, code standards, execution correctness, documentation, and timeliness. Using the predefined rubric criteria, such as design of logic, code standards, program execution, submission timing, and documentation quality, Gemini is prompted to generate structured feedback and approximate scores. This approach allows instructors to obtain AI supported evaluations that align with human-defined grading standards, helping ensure fairness, efficiency, and consistency in programming assessments.

Codex Compiler/Interpreter

Codex compiler that can understand and interpret code. In NEUDev, it serves as a virtual compiler/interpreter, enabling code to be executed directly from the browser. It supports various programming languages and enhances the

assessment experience by simulating how actual code would run and produce output.

Testing Dataset

The researchers gathered a test dataset to evaluate the rubric system of the NEUDev. The dataset is a synthetic dataset gathered from the OPENAI's API. It consists of programming language, question, code, allotted time, and time take. It simulates the submission of the student in the NEUDev's coding assessment platform. The dataset consists of 90 samples. 30 Python scripts, 30 Java, and 30 C# programs. All of these samples are console executable programs. The synthetic dataset does not contain any rubric score or evaluation. That's why the

researcher annotated the samples based on the structure and design logic of the code. Here is a snippet of the testing dataset:


```
{
"programming_language": "Python",
  "question": "Write a formal programming",
  "code": "import random\ndeck = [2,3,4",
  "allotted_time": 1260,
  "time_taken": 1074,
  "metadata": {
    "quality_level": "poor",
    "has_comments": false,
    "has_errors": true,
    "complexity": "basic"
   },
"ai_rubric_evaluation": {
     "designOfLogic": {
     "grade": "Fair",
     "score": 10
    },
     "standards": {
```

```
        "grade": "Good",
        "score": 15
      },
      "programExecution": {
        "grade": "Excellent",
        "score": 20
      },
      "timeTaken": {
        "grade": "Good",
        "score": 15
      },
      "documentation": {
        "grade": "Poor",
        "score": 2
      }
    },
    "totalScore": 62
  }
```


Figure 3.3 Test Data

In the figure 3.3, we can see the different data in every sample. It has programming language, question, code, allotted time in seconds, time-taken, metadata, and the rubric evaluation that is evaluated by the researchers.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the results and findings derived from the evaluation methods conducted in this study. It is followed by a discussion and interpretation of the results' outcomes.

In fulfillment of the first objective: **To provide a programming environment where students can execute the selected programming languages: Python, Java, and C#**, the researchers integrated the Codex API into the NEUDev coding assessment platform. This integration includes adding a WebSocket-based server that accepts raw source code submissions, dispatches them to the Codex API hosted on Railway (making it extensible and easier to customize for the system), and retrieves the compiled output and runtime streams. On the frontend, the development team embedded the Monaco Editor to capture and edit student code, then piped Codex's stout and stderr back into an XtermTerminal component for an authentic in-browser terminal experience.

The Dockerfile provisions Ubuntu 22.04 with Python 3.10.6 (pip 20.1), OpenJDK 11.0.14, and Mono's C# compiler mcs 6.8.0 ensuring consistent runtimes across all three languages. By combining these WebSocket-based services, the Monaco Editor, and XtermTerminal, NEUDev enables both students and instructors to write, compile, and debug Python, Java, and C# programs entirely within the platform.

NEUDev's Code Execution

```
START

IF student_clicks("Run Code") THEN

    dispatchToWebSocketServer(request)

    sessionDir = createSessionDirectory()

    writeCodeFiles(sessionDir, submittedCode)

  compileCommand, executeCommand =
  commandMap(submittedCode.language)

    compileResult = compileCode(sessionDir, compileCommand)

    IF compileResult.success THEN
        executionResult = executeCode(sessionDir,
executeCommand)

        streamToClient(executionResult.output,
executionResult.error)

        sendExitCodeToClient(executionResult.exitCode)
    ELSE
        streamToClient(compileResult.error, null)
        sendExitCodeToClient(compileResult.exitCode)
    END IF

    deleteSessionDirectory(sessionDir)

END IF

END
```

Pseudocode 1.1 NEUDev's Code Execution

Pseudocode 1.1 illustrates the end-to-end lifecycle of a code submission in NEUDev's compiler service. Once a student clicks Run Code, the request is handed off to our WebSocket-based server, which first creates an isolated session directory and writes the submitted source files to disk. The service then invokes our commandMap helper to select the correct compile and execution

commands, compiles the code if necessary, and launches it inside a pseudo-terminal. As the

program runs, its standard output and error streams are relayed back in real time to the browser; when execution finishes, the server sends the final exit code and tears down the session directory.

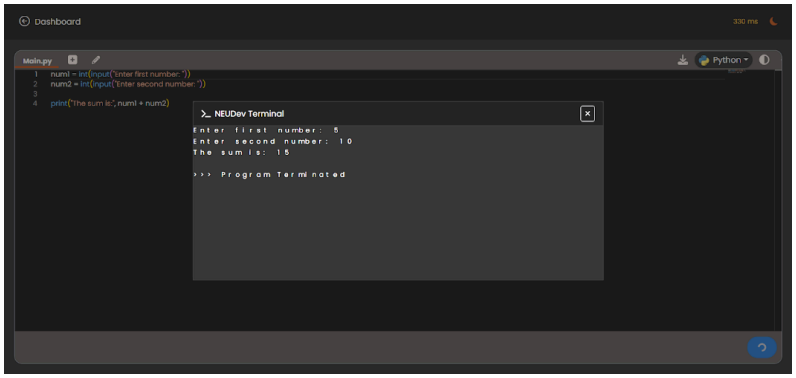


Figure 4.1 Python Execution in Sandbox

In fulfillment of the second objective: To integrate CodeSpy's AI code detection capability into the NEUDev platform for identifying AI-generated code submissions, the researchers implemented Python code execution within the NEUDev coding assessment sandbox. Figure 4.1 demonstrates the Python environment in action, where a user inputs two numbers and the program returns their sum. This execution happens inside a controlled terminal called **NEUDev Terminal**, and is part of the platform's support for running Python code securely and interactively during assessments.

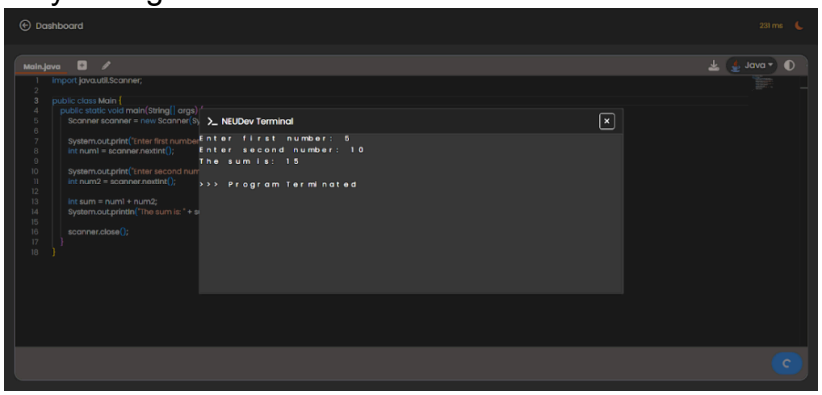


Figure 4.2 Java Execution in Sandbox

Figure 4.2 shows the execution of a similar task using Java code inside the NEUDev sandbox. Students can write, run, and test Java code through the NEUDev Terminal. The backend captures the code and results, enabling both learning and assessment. This Java execution environment is seamlessly integrated into the platform and is also compatible with the AI code detection module powered by CodeSpy.

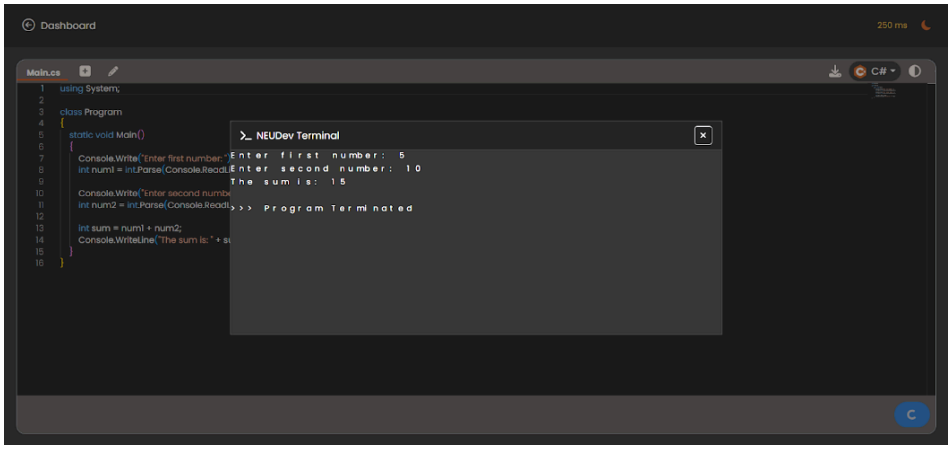


Figure 4.3 C# Execution in Sandbox

As shown in Figure 4.3, C# is also supported within the NEUDev sandbox. This figure displays a simple C# program that reads two numbers from the user and outputs their sum, running inside the same NEUDev Terminal. The support for multiple programming languages, including C#, is part of the NEUDev platform's core feature set, which works in tandem with the CodeSpy API to detect AI-generated submissions regardless of the language used.

In fulfillment of the second objective: **To integrate CodeSpy's AI code detection capability into the NEUDev platform for identifying AIgenerated code submissions**, the researchers integrated the CodeSpy API to the NEUDev

coding assessment platform. The integration includes adding a backend service in Laravel that accepts source codes, sends the code to the API services, and retrieves the data from the CodeSpy about the score of the AI Generated Code. The developers and researchers also included the score into the display of the activity records of the teacher per activity. This allows the teacher user to see who among the student used an AI generated code in their submissions. The process of the detection of AI Code is discussed below:

CodeSpy AI Code Detection

```
BEGIN CodeSpyIntegration

RECEIVE submittedCode FROM Student

CALL finalizeSubmission(submittedCode)

SET savedCode = GET_CODE_FROM_DATABASE()

CALL processAIDetection(savedCode)

SEND savedCode TO CodeSpyAPI

RECEIVE responseJSON FROM CodeSpyAPI

SET aiScore = responseJSON.score
SET metadata = responseJSON.sourceInfo

CALL SaveAIScoreToDatabase(aiScore, metadata)

DISPLAY aiScore, metadata TO TeacherUI

END CodeSpyIntegration
```

Pseudocode 1.2 CodeSpy AI Code Detection

In the figure 1.2, we see how the CodeSpy works in the NEUDev platform. It starts when the student submits a source code from one of the assessments in the NEUDev. The finalizeSubmission() method will save the code in the database. Now, the processAIDetection() method comes in to get the saved code

in the database and sends it to the CodeSpy API service. After the code is sent to the API service, the CodeSpy replies with a JSON with the score of the AI generated code and the other data about the source code. The controller now saves the score and data into a new table that can be retrieved to the user interface of the teacher.

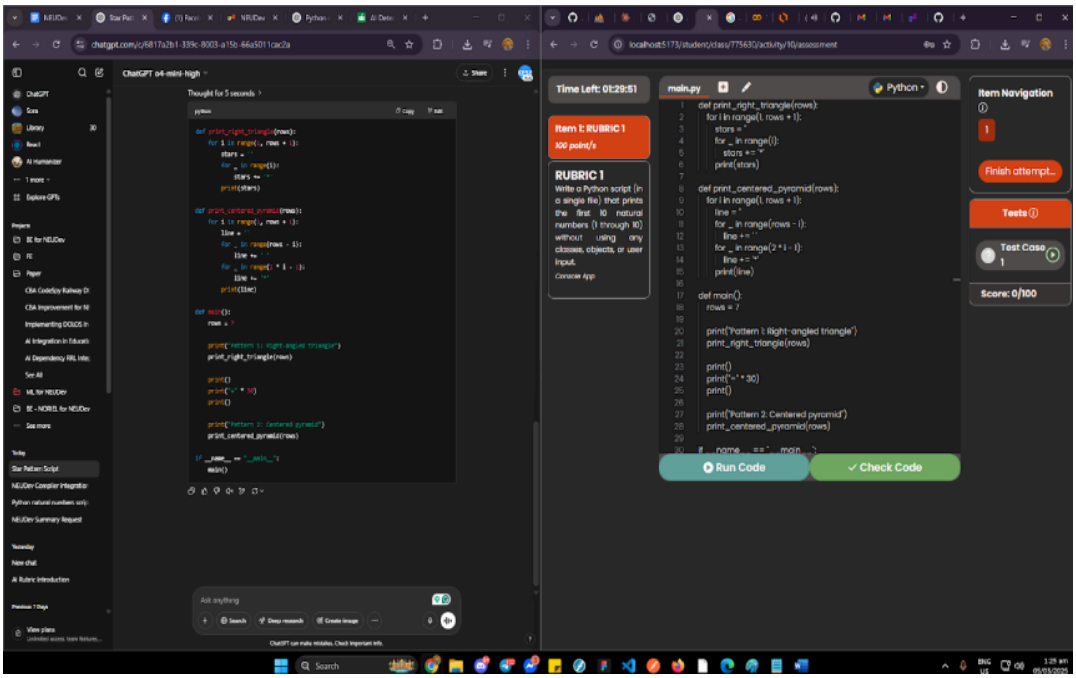


Figure 4.4 Chat GPT Generated Submitted Code

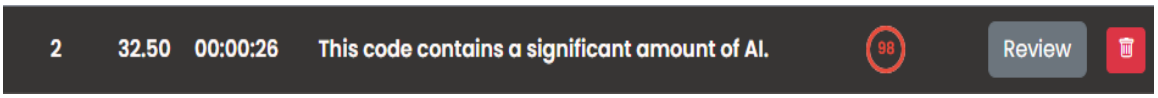


Figure 4.5 AI Code Prediction in the Teacher's Record of Student Submission Table

Figures 4.4 and 6.5 show the integrated AI Code Detection in the NEUDev. In the figure 4.4, we can see the student submitted a Python Code copied from the Chat GPT's output. Once it is submitted, the source code of the student will be processed in the model and will be predicted whether it is an AI Generated Code or Human Written Code. In the figure 4.5, we can see in the

activity class record that the student submission was flagged as AI Generated Code with a 98.0% percentage of AI Generated Code.

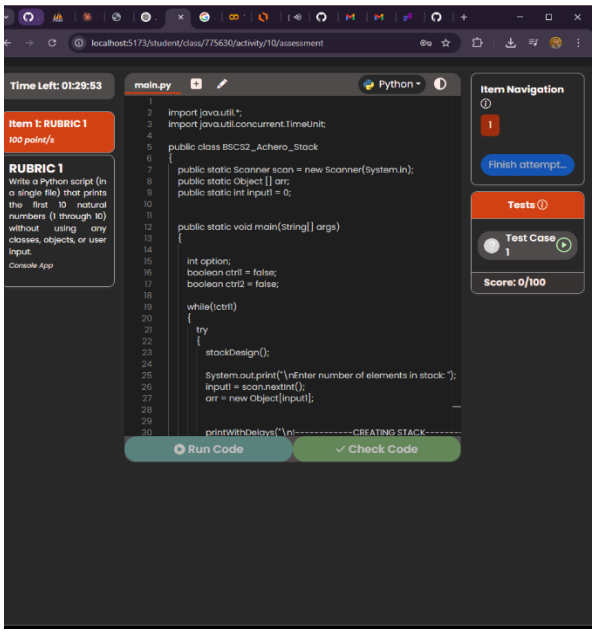


Figure 4.6 Student Human Submitted Code

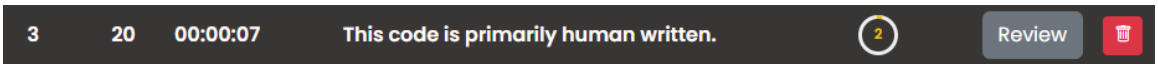


Figure 4.7 Human Prediction in the Teacher's Record of Student Submission Table

Figures 4.6 and 4.7 shows the integrated AI Code Detection in the NEUDev. In the figure 4.6, we can see the student submitted a human written Java Code of tally in attendance in a seminar. Once it is submitted, the source code of the student will be processed in the model and will be predicted whether it is an AI Generated Code or Human Written Code. In the figure 4.7, we can see in the activity class record that the student submission was flagged as human written with only a 2.0% percentage of AI Generated Code.

In fulfillment of the third objective: T**o integrate DOLOS into NEUDev for detecting code similarity among student submissions and assisting**

instructors in identifying potential cases of plagiarism, the researchers and developers integrated the DOLOS Similarity API to the NEUDev Coding Assessment Platform. In the integration, it includes creating backend services in the Laravel 11 and an API UI that links to the website of the DOLOS. The backend includes a service that gets the file submissions of the students in an activity and sends them to the DOLOS via API. The API responds with a link to the results of the similarity detection. The link is now embedded in the the UI of the activity submissions for the teacher to see and review the similarities of the students' submissions. The process of the similarity detection among students' code submissions is discussed below:

DOLOS Similarity System

```
START

RECEIVE submittedCode FROM Student

CALL finalizeSubmission(submittedCode)

RECEIVE similarityRequest FROM Instructor

SET submissionsList = FETCH ActivitySubmissions FROM Database

SET groupedSubmissions = GROUP submissionsList BY
programmingLanguage

FOR EACH languageGroup IN groupedSubmissions
CREATE zipArchive FROM languageGroup
ADD zipArchive TO archiveList
END FOR

FOR EACH archive IN archiveList
SEND archive TO DolosAPI
END FOR

SET reportURLs = RECEIVE feedbackURLs FROM DolosAPI

CALL SaveDolosURLsToDatabase(reportURLs)

SET displayReports = FETCH reportURLs FROM Database
```

```
DISPLAY displayReports TO InstructorUI
END
```

Pseudocode 1.3 DOLOS Similarity System

In Pseudocode 1.3, upon submission, NEUDev's backend persists each student's code into the activity_submissions table. When an instructor subsequently requests a similarity analysis, the system retrieves all relevant submissions, organizes them by programming language, and generates a separate ZIP archive for each language group. These archives are transmitted to the DOLOS API via the DolosService::createReport() method, which returns a shareable report URL for each dataset. NEUDev then records these URLs in the activity_similarity_reports table and presents them within a modal dialog in the instructor's dashboard, enabling immediate access to detailed, language-specific similarity reports without leaving the platform.

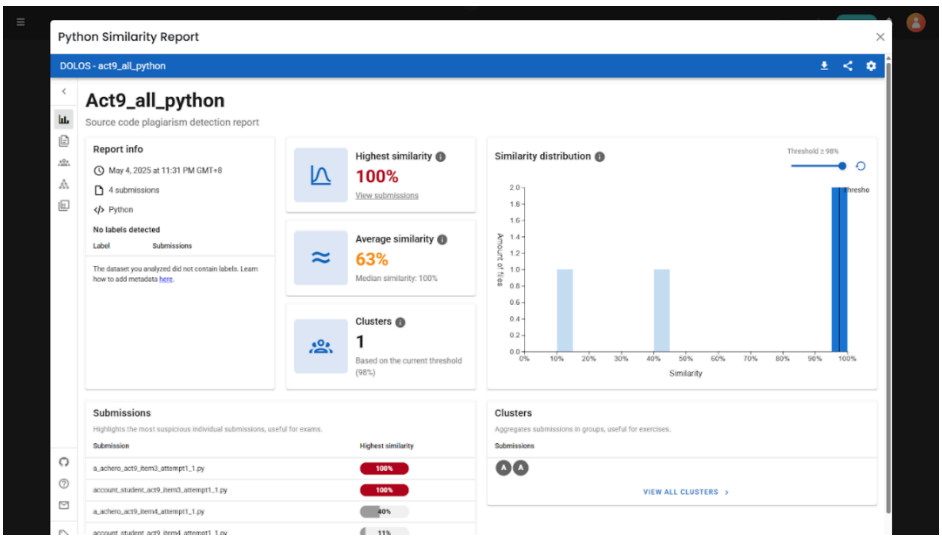


Figure 4.8 Overview of the statistic of the similarity in a python activity

Figure 4.8 shows a sample similarity detection report for a Python coding activity utilizing the DOLOS plagiarism detection tool. The report summarizes how similar student submissions are to one another. In this example the highest

similarity found between two submissions is 100%, which means they are exactly or almost exactly the same. The average similarity across all submissions is 63%, suggesting that many students submitted code with noticeable similarities. The system also detected one cluster of highly similar submissions, based on a set threshold of 95%. The graph on the right shows how many submissions fall within different similarity ranges. A large number of submissions are in the 90–100% range, indicating possible cases of copying. This report helps teachers identify which students may have copied each other's code and take the necessary steps to address academic integrity issues.

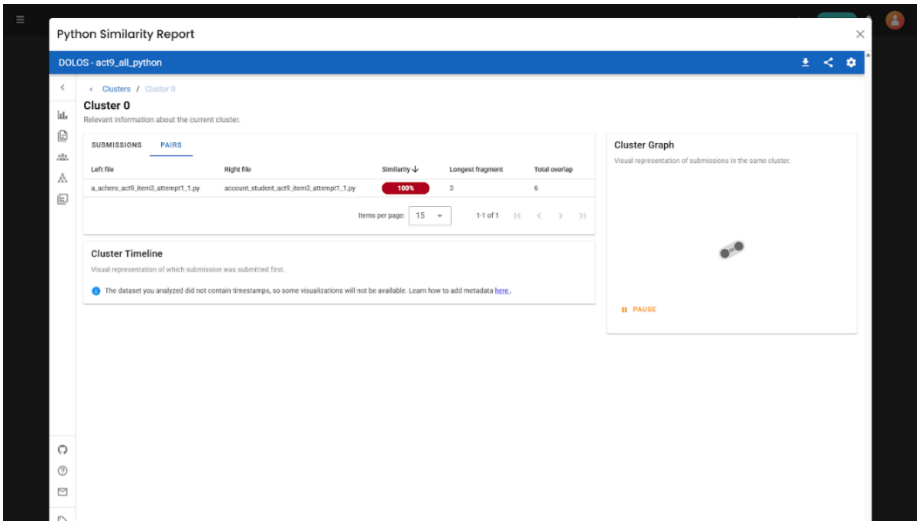


Figure 4.9 Pairs of similarity in code submissions

This image shows a similarity detection report for a Python coding activity titled "Act9_all_python", generated using the Dolos tool. The report analyzes multiple student code submissions and highlights potential plagiarism or collaboration. The report indicates that the highest similarity between any two submissions is 100%, which means they are identical or nearly identical. The average similarity among all submissions is 63%, suggesting a moderate level of

overall similarity in the class. A single cluster was identified, which groups submissions with very high similarity (above a 98% threshold), indicating a strong chance of copied work. On the right, the similarity distribution graph shows that most submissions fall within the highest range (90–100%), further supporting the presence of possibly copied codes. The submissions list at the bottom highlights the specific files and their similarity percentages, allowing instructors to easily identify suspicious pairs.

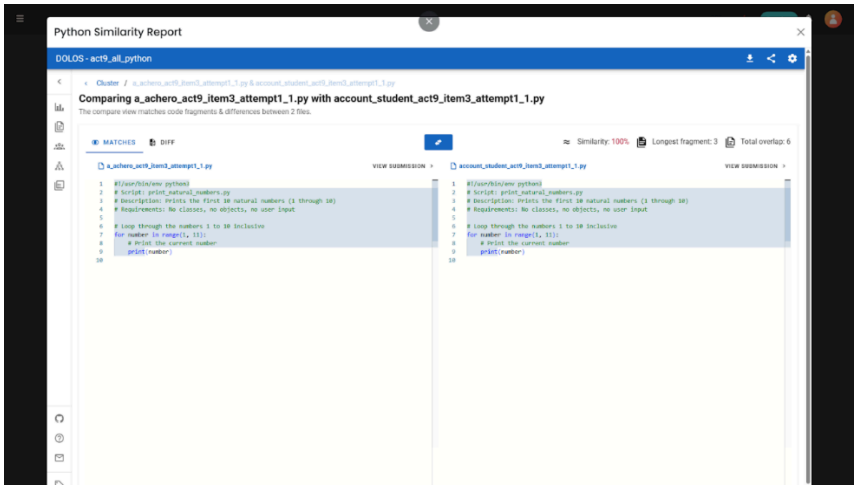


Figure 5.1 Highlights of code in pairs of similarity in two code submissions

This image shows a similarity report from DOLOS, a tool used to check code submissions. It compares two Python files and shows they're 100% similar, meaning the code is basically the same. The files are grouped in what's called "Cluster 0" because of how similar they are. On the right, there's a simple graph showing the two files are connected. The timeline section says timestamps weren't available, so it can't tell which one was submitted first.

In fulfillment of the fourth objective: **To implement a code evaluation system that uses test-case-based scoring framework and rubric to provide students with quantifiable scores and targeted feedback on their code submissions**, the researchers integrated the NEUDev coding assessment platform. The integration includes adding backend services and frontend components. The rubric system is an AI powered system that utilizes the power of the Gemini LLM. The test-case-based scoring and rubrics are discussed below.

Test Case and Rubrics System

```
START
RECEIVE checkCodeTrigger FROM Student
CALL CompilerService WITH submittedCode
SET latestOutput = CompilerService.output
SET expectedOutput = RetrieveExpectedOutput()
IF latestOutput == expectedOutput THEN
    SET testCaseResult = "Pass"
ELSE
    SET testCaseResult = "Fail"
END IF
UPDATE testCaseResult IN Database
IF CodeRestriction IS Enabled THEN
    SET runCount = GET NumberOfRuns FROM Database
    DEDUCT Score BASED ON runCount
END IF
SET testCaseScore = CALCULATE TestCaseScore(testCaseResult)
SEND questionData, code, timeAllotted, timeTaken TO GeminiAPI
RECEIVE rubricEvaluationJSON FROM GeminiAPI
SET rubricScore = rubricEvaluationJSON.score
SET itemScore = testCaseScore + rubricScore
ACCUMULATE itemScore INTO totalScore
DISPLAY totalScore AND rubricFeedback TO StudentUI
END
```

Pseudocode 1.4 Test Case and Rubrics System

In Pseudocode 1.4, the scoring process begins when a student triggers the “Check Code” action. This initiates the compilation of the submitted code, followed by a result comparison between the latestOutput, the user’s output, and the expectedOutput. This determines whether the test case is passed or failed

and updates the test case result accordingly. Next, the system computes the item score. If code restriction is enabled, score deduction based on the number of runs will be applied; otherwise, it is skipped. The rubric evaluation is then performed using the Gemini API, which receives data such as the question, code, time allotted, and time taken. The API responds with a JSON object containing rubric evaluation results, including the rubric score. Finally, the item score is calculated by summing the test case score and the rubric score: Test Case Score + Rubric Score = Item Score. The total score is accumulated across items and displayed on the user interface.

NEUDev's Rubrics

This is the programming rubric of every programming activity in the NEUDev Coding Assessment Platform. The rubric was checked by Dr. Marc P. Laureta, a programming professor at the New Era University, College of Informatics and Computing Studies. See the rubric below:

Table 3.4 NEUDev's Rubrics

Criteria	Excellent	Good	Fair	Poor	Score	Comments
Design of Logic (30 pts)	Program is logically well-designed (30)	Program has slight logic errors that do not significantly affect the results (20)	Program has significant logic errors (10)	Program is incorrect (5)		
Standards (20 pts)	Program is stylistically well-designed (20)	Few inappropriate design choices (i.e poor variable names, improper indentation (15)	Several inappropriate design choices (10)	Program is poorly written (5)		
Program Execution (20 pts)	Program executes correctly with no syntax or runtime errors (20)	Program executes with minor syntax or runtime errors (15)	Program executes with a minor (easily fixed) error (10)	Program does not execute (5)		

Time Taken (20 pts)	Submitted well ahead of time (20)	Submitted slightly early (15)	Submitted with little time remaining (10)	Did not finish (5)		
Documentation (10 pts)	Program is well documented with comments (10)	Missing one required comment (8)	Missing two or more required comments (5)	Most or all documentation are missing (2)		
Total Score:					100	

The table above is a grading-rubric that the AI uses as a basis for evaluating a computer programming assessment. There are 5 evaluation criteria, each having their own corresponding value and showcases different levels of performance (Excellent, Good, Fair, Poor) and also corresponding scores respectively.

Criteria Categories:

1. **Design of Logic (30 points)**
    - **Excellent (30 pts):** *Evaluates how logically sound and structured a student's program is*
    - **Good (20 pts):** *Contains slight logic errors that don't majorly affect results*
    - **Fair (10 pts):** *There are some significant logic errors*
    - **Poor (5 pts):** *The program logic is significantly flawed or incorrect*
2. **Standards (20 pts)**
    - **Excellent (20 pts):** *The program is clean and well-designed*
    - **Good (15 pts):** *There are some poor design choices (e.g, bad variable name or improper indentation)*

- **Fair (10 pts):** *Several poor design choices*
- **Poor (5 pts):** *Code is poorly written*

3. **Program Execution (20 pts)**
    - **Excellent (20 pts):** *The program has no syntax or runtime errors*
    - **Good (15 pts):** *Minor syntax or runtime errors*
    - **Fair (10 pts):** *Minor errors that can be easily corrected*
    - **Poor (5 pts):** *Program doesn't run at all*
4. **Time Taken (20 pts)**
    - **Excellent (20 pts):** *Submitted ahead of the deadline*
    - **Good (15 pts):** *Submitted slightly early*
    - **Fair (10 pts):** *Submitted with little time remaining*
    - **Poor (5 pts):** *Did not finish on time*
5. **Documentation (10 pts)**
    - **Excellent (20 pts):** *Properly documented with appropriate comments*
    - **Good (15 pts):** *Missing one required comment*
    - **Fair (10 pts):** *Missing two or more required comment*

- **Poor (5 pts):** *Most or all of the documentation are missing*

The *Scoring column* is for the AI to input the score the student earned in each category and the *Comments column* is a dedicated space to write specific feedback or justification for the given score. The *last row* with the *Total Score* sums up the total score of the student out of 100 points.

Here is the sample response of the Gemini API in the rubrics:


```
{
    "designOfLogic": {
      "grade": "Excellent",
      "score": 30,
      "feedback": ""
    },
    "standards": {
      "grade": "Excellent",
      "score": 20,
       "feedback": ""
    },
    "programExecution": {
      "grade": "Excellent",
      "score": 20,
      "feedback": ""
    },
    "timeTaken": {
      "grade": "Excellent",
      "score": 20,
      "feedback": ""
    },
    "documentation": {
      "grade": "Excellent",
      "score": 10,
      "feedback": ""
    },
    "totalScore": 100,
    "feedbackSummary": ""
}
```


Figure 5.2 Gemini API

The figure 5.2 includes the sample output of the Gemini API in the rubrics of the NEUDev. The API returns a JSON that consists of the grade given by the

AI based on the student's code. It has different criteria, and the results and feedback on the code.

Scoring Formulas

Let

P - total points assigned to one item

N - number of test cases for that item

R - number of "Check Code" runs the student used

d - penalty (in points) deducted per run

δi ∈{0,1} - indicator that test case i passed (1) or failed (0)

The framework evenly divides an item's total points P across its N test cases, so each individual test carries p points.

$$P = \frac{P}{N}$$ ***Eq 4.1*** *Points per test case*

Remaining points after run penalties

Every "Check Code" run costs a fixed penalty d, and the system subtracts d times the number of runs R from P, ensuring the result never falls below zero.

$$r = max(P - dR, 0)$$ ***Eq 4.2*** *Remaining points after run penalties*

Fraction of tests passed

Measures the share of test cases the student's code passed counting each passed test ($\delta\ i$ = 1 δ i =1) and dividing by the total N.

$$f = \frac{\sum_{i=1}^{N} \delta_i}{N}$$

Eq 4.3 *Fraction of tests passed*

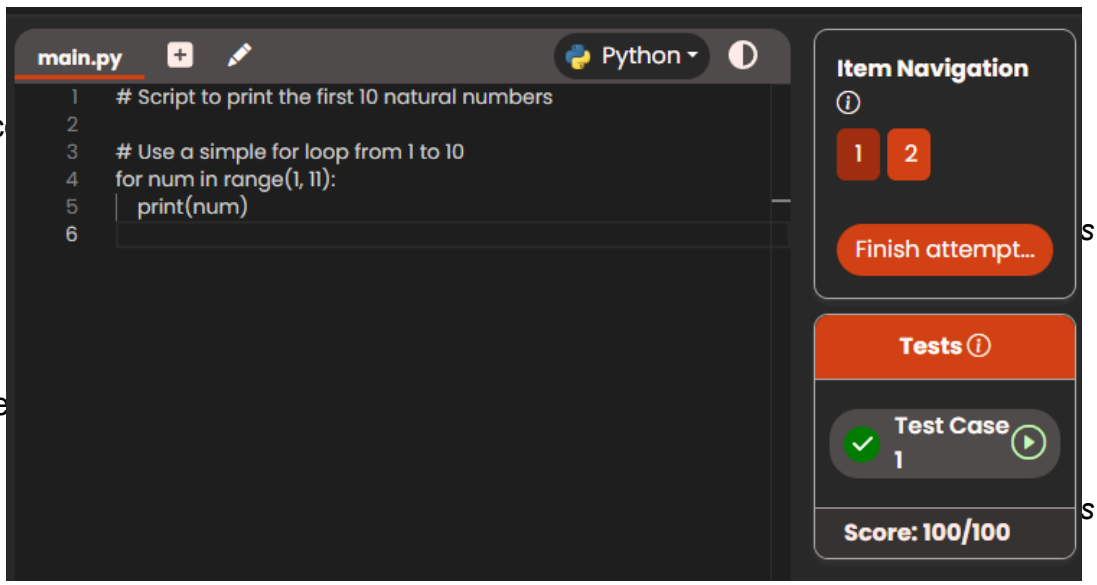
Score for one item

The framework takes the remaining points r and scales them by the fraction f of tests passed, yielding the final score for that item.

$$S_{item} = r \times f = (P - dR) \times \frac{\sum_{i=1}^{N} \delta_i}{N}$$

Eq 4.4 *Score for one item*

proc



s

asse

s

Figure 5.3 Test Case Scoring

Figure 5.3 shows a sample scoring screen from a coding platform. The code on the left is a simple Python script that prints the first 10 natural numbers using a for-loop. It includes basic comments and uses range (1, 11) to go from 1 to 10. On the right side, the system shows the test results. It passed the test case and got a perfect score of 100 out of 100. This means the code works correctly based on the task and test provided.

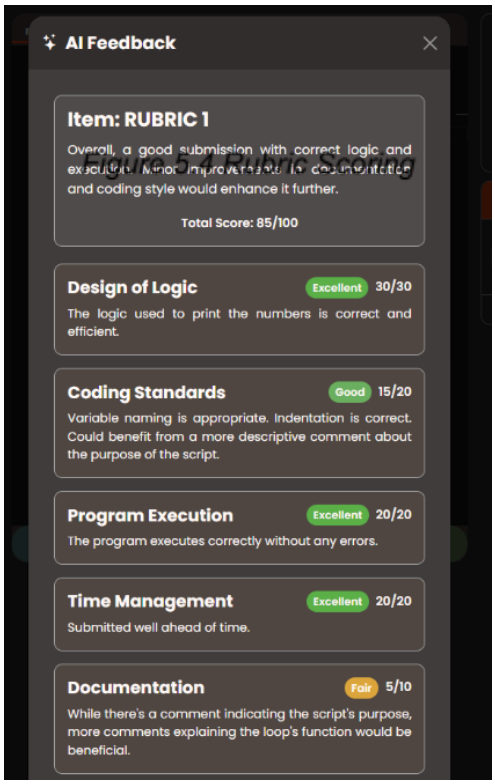


Figure 5.4 Rubric Scoring

The image shows an AI-generated rubric scoring for a programming task labeled “RUBRIC 1,” with a total score of 85 out of 100. The feedback highlights that the submission had correct logic and execution, with only minor

improvements needed in documentation and coding style. The **Design of Logic** was rated as Excellent, earning a full 30/30 points for being correct and efficient. **Coding Standards** received a good rating with 15/20 points; although variable naming and indentation were appropriate, the script lacked a more descriptive comment explaining its purpose. **Program Execution** and **Time Management** both received Excellent ratings with full scores of 20/20, indicating the code ran without errors and was submitted early. However, **Documentation** scored only 5/10 with a Fair rating, as it contained minimal comments and lacked explanations about the loop's function.

In fulfillment of the fifth objective: **To conduct performance testing of the NEUDev platform through stress testing and pilot testing in real-world classroom settings to assess its scalability, responsiveness, and stability under varying loads of concurrent users**, the developers used K6 for stress testing and pilot tested the NEUDev in an exam of two sections of the course Fundamentals of Programming.

The stress testing that was run in the K6 includes the following results:

Thresholds

Table 4.1 Thresholds

Checks	**Status**	Details
checks	Passed	rate = 99.94 %
checks {group:::01b_EnrollClass}	Passed	rate = 100.00 %
connection_errors	Passed	count = 0 (< 150)
http_req_duration p(90)<5000	Passed	p(90)=2.57 s (l, 99 %: 27 560 ✓ / 257 ✗)
ws_error_rate < 10 %	Passed	0.92 %
checks	Passed	rate = 99.94 %

Table 4.1 shows that the system met all the expected performance criteria. The overall check rate passed at 99.94%, meaning almost all requests were successful. The specific check for the "Enroll Class" feature had a perfect 100% pass rate. There were no connection errors recorded during the test, staying well below the allowed limit of 150. The p90 response time was 2.57 seconds, which is within the 5-second threshold, and 99% of requests were successful in that group. Lastly, the WebSocket error rate was only 0.92%, which is much lower than the 10% limit.

Table 4.2 Total Results

Metric	Avg	Min	Med	Max	p(90)	p(95)	Rate (/s)	Value
assessment_load_time	1 703 ms	37 ms	922 ms	16 288 ms	4 553 ms	6 060 ms	–	–
connection_errors	0	–	–	–	–	–	0/s	0
enroll_time	831 ms	13 ms	279 ms	15 054 ms	2 418 ms	3 579 ms	–	–
http_errors	–	–	–	–	–	–	0.000504/s	1
http_requests	–	–	–	–	–	–	169.19/s	335 466
checks_total	–	–	–	–	–	–	239.62/s	475 081
checks_succeeded	–	–	–	–	–	–	–	99.94% (474 822 / 475 081)
checks_failed	–	–	–	–	–	–	–	0.05% (259 / 475 081)
login_time	891.85 ms	13 ms	347 ms	121 502 ms	2 555 ms	3 670 ms	–	–
run_code_time	17 737.86 ms	205 ms	3 724.5 ms	64 246 ms	59 256.2 ms	60 346.05 ms	–	–
save_progress_time	855.63 ms	15 ms	313 ms	15 864 ms	2 426 ms	3 556 ms	–	–
submit_time	1 582.40 ms	271 ms	1 313 ms	13 425 ms	3 404 ms	4 608 ms	–	–

The table 4.2 summarizes the results of the 30-minute load test on the NEUDev Coding Assessment Platform using 1000 virtual users. Key user actions such as login, enrollment, fetching assessments, running code, saving progress, and submitting assessments were measured.

On average, logging in took around 892 ms, enrolling took 831 ms, and submitting took 1582 ms. The run code action had the highest average time at 17.7 seconds, peaking at over 64 seconds, which is expected due to backend code execution.

Assessment load and save progress times remained under 2 seconds on average. Importantly, there were zero connection errors and only one HTTP error recorded throughout the test. Out of over 475,000 checks, 99.94% succeeded, demonstrating that the platform handled the high traffic well and maintained stability across all key features.

Table 4.3 HTTP Details

Metric	Avg	p(90)	p(95)	Max
http_req_duration	919 ms	2.57 s	3.70 s	1 m 0 s
http_req_duration {expected_response}	962 ms	2.64 s	3.77 s	15.86 s
http_reqs	335 466	169.20/s	–	–

The table 4.3 reflects key HTTP performance metrics during the K6 load test of the NEUDev Coding Assessment Platform. The average HTTP request duration was 919 milliseconds, with 90% of requests completing within 2.57 seconds and 95% within 3.70 seconds. The maximum observed duration for a request reached up to 1 minute, though such cases were rare. For requests that returned the expected response, the average duration was slightly higher at 962 milliseconds, with a 95th percentile of 3.77 seconds and a maximum of 15.86 seconds. Overall, the platform handled a total of 335,466 HTTP requests at a

steady rate of approximately 169 requests per second, indicating stable performance under high concurrent usage.

Table 4.4 Execution

Metric	Value
iteration_duration	avg = 9.71 s • p(90)=18.33 s • p(95)=46.61 s • max = 2 m 1 s
iterations	111 617 (56.30 /s)
vus	min = 1 • max = 1 000 • end = 2
vus_max	1 000

Metric	Value
ws_connecting	avg = 17.23 s • p(90)=58.21 s • p(95)=59.12 s • max=1 m 0 s
ws_error_rate	0.92 % (257 errors / 27 826 sessions)
ws_errors	257 (0.13 /s)
ws_msgs_received	55 141 (27.81 /s)
ws_msgs_sent	27 569 (13.91 /s)
ws_requests	27 983 (14.11 /s)
ws_session_duration	avg = 18.12 s • p(90)=59.54 s • p(95)=1 m 0 s
ws_sessions	27 826 (14.03 /s)

Table 4.4 shows that the WebSocket's performance of the NEUDev Coding Assessment Platform during the test was stable and reliable. On average, it took about 17 seconds to establish a WebSocket connection, with most users connecting within 58 seconds and sessions lasting around 18 seconds on average. Out of over 27,000 sessions, only 0.92% experienced errors, which is well within acceptable limits.

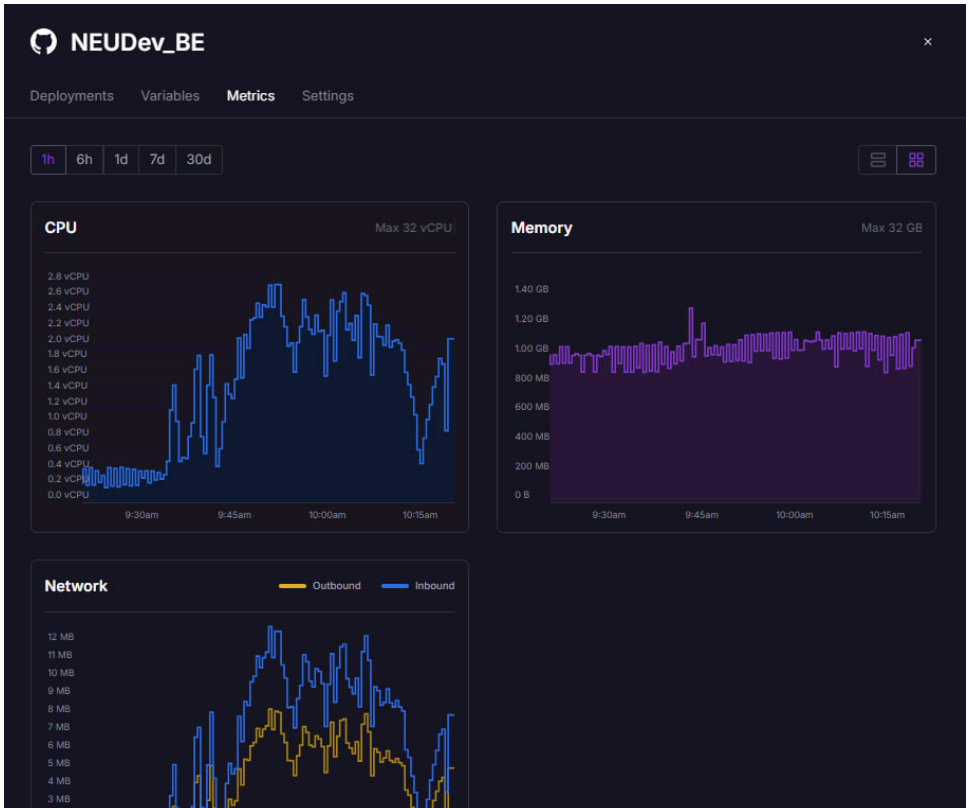
The system handled around 28,000 WebSocket requests, with messages being sent and received smoothly at consistent rates. Overall, the platform managed real-time interactions effectively even under heavy simulated student activity.

The NEUDev Coding Assessment Platform on Railway was put through a 30-minute k6 stress test with 1,000 simultaneous virtual student users. Over that period,

it handled more than 335 466 HTTP requests and 27 826 WebSocket sessions. Median HTTP response duration was 919 ms with a 90th percentile of 2.57 s, while key actions such as class enrollment, activity loading, progress saving, and final submission, averaged under two seconds each.

The WebSocket-based code execution step ran in about 17.23 s on average (p90 ≈ 59.21 s) but still maintained a sub-1 % error rate.

Overall, over 99.9 % of all checks passed, demonstrating that NEUDev remains stable, responsive, and highly scalable under heavy, end-to-end student workloads.



users

the NE . CPU

usage U and

averag y held

steady icating

effecti ked at

11–12 eriods,

then fell back as load eased.

Figure 5.6 NEUDev_Compiler

In Figure 5.6, the NEUDev Compiler service handled WebSocket-based code execution by ramping the CPU to about 5 vCPUs under peak demand and averaging 4–4.5 vCPUs once the traffic stabilized. Its memory footprint grew from roughly 200 MB at test start to about 1.3 GB by the end, reflecting temporary heap allocations during runs but no uncontrolled accumulation. Network I/O remained modest, with inbound peaks under 700 KB/s and outbound around 450 KB/s, matching the small size of WebSocket messages. Together these profiles confirm that both services scales linearly under heavy student workloads while maintaining stable CPU, memory, and network usage and leaving plenty of headroom for higher concurrency. On the pilot testing for the two sections of Fundamentals of Programming Course includes using of Google Forms to gather their insights while using the NEUDev in their exam. Also, the CPU and memory usage are monitored while the exams are ongoing. Here are the data of the pilot testing:

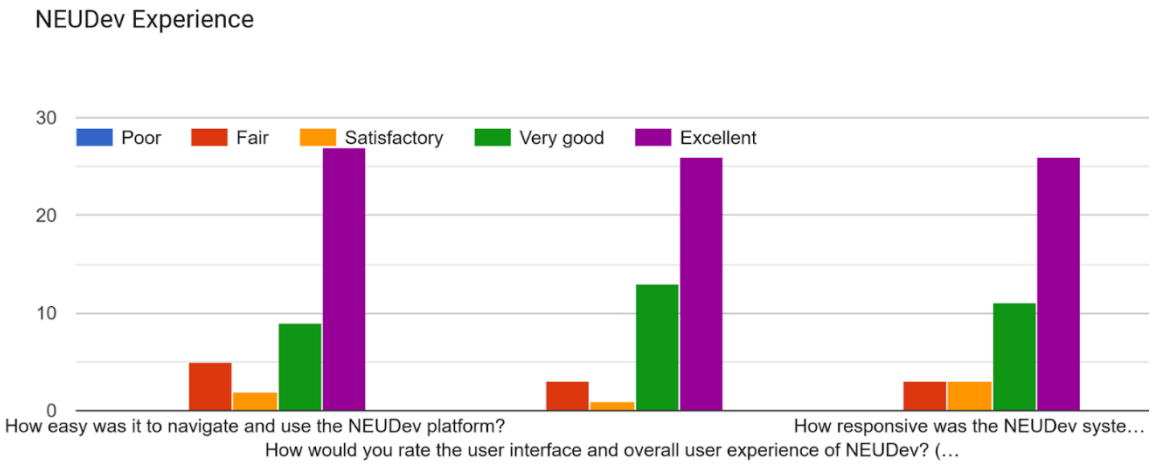


Figure 5.7 Google Forms Summary of Student Feedback on NEUDev Platform Experience

To gain insight into the user experience of NEUDev, a Google Forms survey was conducted among 43 student users to evaluate their interaction with the system. The results showed that the majority of students found the platform intuitive and user-friendly. Specifically, 27 respondents rated the ease of navigation and usage of NEUDev as “Excellent”, while 10 rated it as “Very Good”. Only a small minority gave ratings of “Fair” or “Poor”, indicating a generally positive reception of the system’s interface and accessibility.

When asked about the user interface and overall experience, 27 out of 43 students rated NEUDev as "Excellent", followed by 13 who rated it as "Very Good." This confirms the platform’s effectiveness in delivering a seamless and engaging user experience. Additionally, in terms of system responsiveness during use such as running code, checking outputs, and submitting assessments 26 respondents rated it "Excellent", while 11 gave it a "Very Good" rating. These results reinforce NEUDev’s value as an efficient and well-performing tool for

coding assessments, meeting the usability expectations of its primary users, students of New Era University.

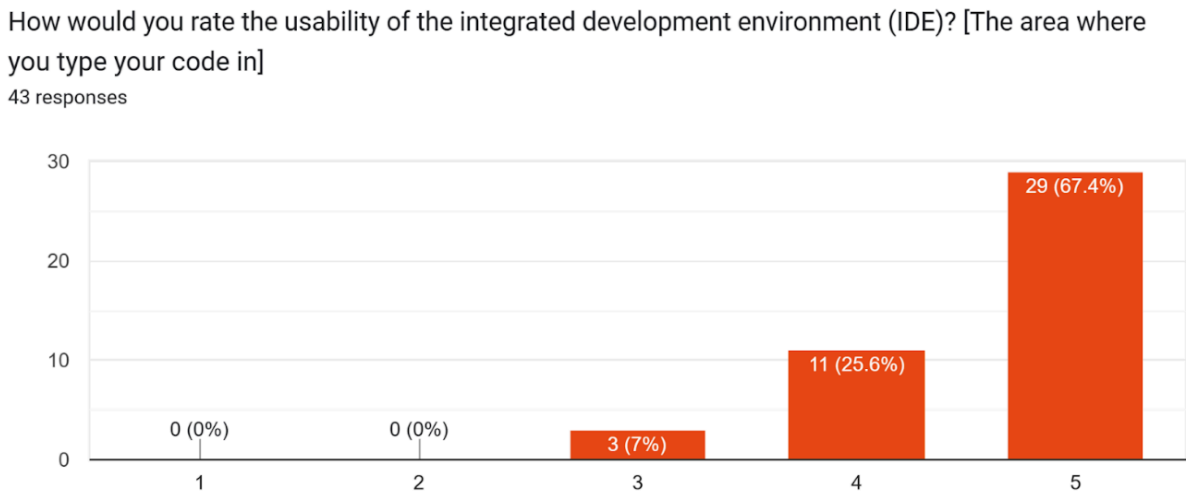


Figure 5.8 Student Ratings on the Usability of the NEUDev Integrated Development Environment (IDE)

In a separate question focusing on the coding area of NEUDev, students were asked to rate the usability of the integrated development environment (IDE), the part of the platform where they type and test their code. Out of 43 respondents, 29 students (67.4%) gave the IDE the highest rating of 5, while 11 students (25.6%)

rated it a 4. Only 3 respondents (7%) gave a neutral score of 3, and none rated it poorly. The overall average rating was 4.60 out of 5.

These results highlight the effectiveness and user-friendliness of the NEUDev IDE, suggesting that students find it comfortable and intuitive to use during coding assessments.

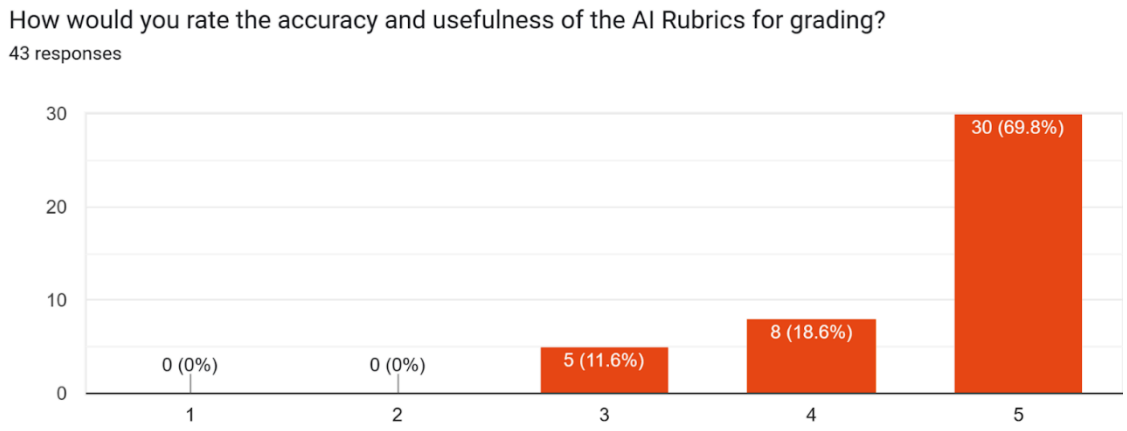


Figure 5.9 Student Ratings on the Accuracy and Usefulness of AI Rubrics in NEUDev

To evaluate how students perceived the AI-powered rubric grading system in NEUDev, a survey was conducted involving 43 respondents. When asked about the accuracy and usefulness of the AI Rubrics in grading their code submissions, a significant portion of students gave favorable responses. Specifically, 30 out of 43 students (69.8%) rated the feature a perfect 5, while 8 students (18.6%) rated it a 4, indicating a high level of satisfaction. Only 5 students (11.6%) provided a neutral rating of 3, and no respondents rated it poorly (1 or 2). The average rating.

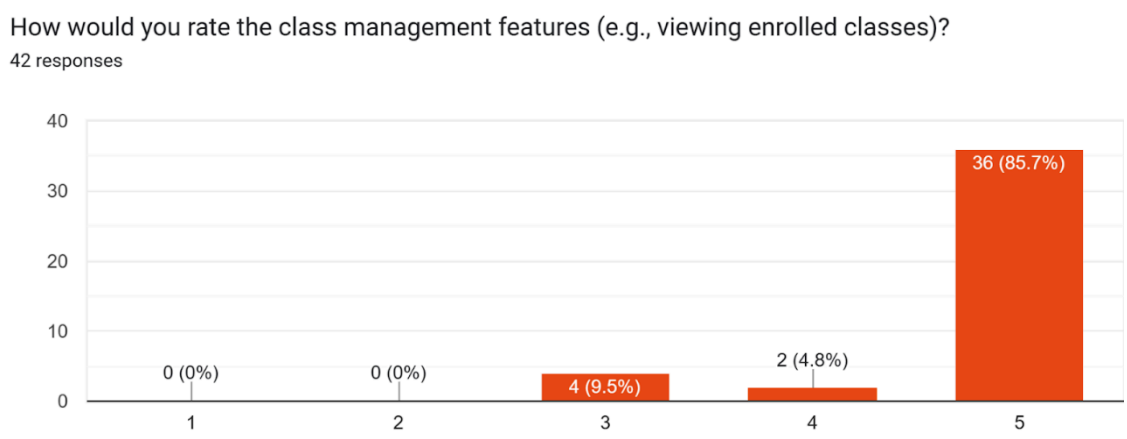


Figure 6.1 Student Ratings on the Class Management Features of NEUDev

To measure the effectiveness of NEUDev's class management features such as viewing enrolled classes and managing class participation students were asked to rate their experience. Out of 42 respondents, 36 students (85.7%) gave the highest rating of 5, and 2 students (4.8%) rated it a 4. Meanwhile, 4 students (9.5%) gave a neutral rating of 3, and none rated the feature poorly (1 or 2). The average rating recorded was an impressive 4.76 out of 5, indicating that the majority of users found the class management functionalities smooth, accessible, and highly effective.

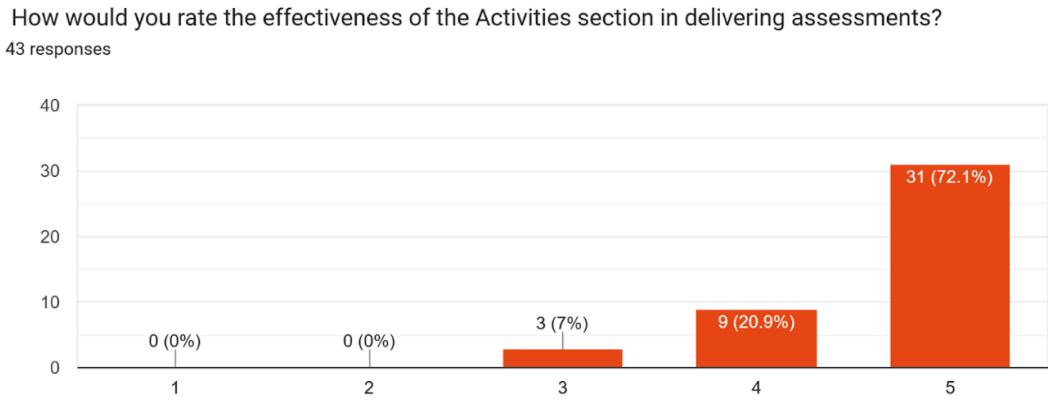


Figure 6.2 Student Ratings on the Effectiveness of the Activities Section in NEUDev

To evaluate the NEUDev platform's capability in delivering programming assessments, students were asked to rate the effectiveness of the Activities section. Among the 43 respondents, 31 students (72.1%) rated it a 5, and 9 students (20.9%) rated it a 4, showing a strong consensus toward its effectiveness. Only 3 students (7%) gave a neutral rating of 3, and no respondents gave scores of 1 or 2. The average rating was 4.65 out of 5, signifying a high level of satisfaction.

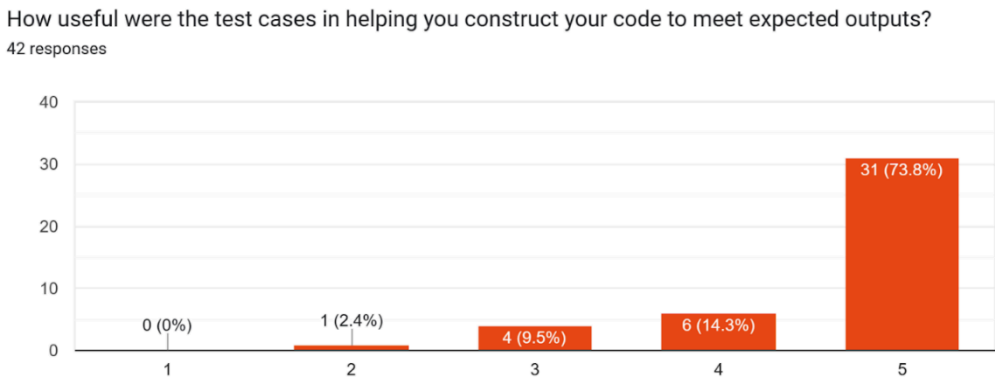


Figure 6.3 Student Ratings on the Usefulness of Test Cases in NEUDev

Students were asked to evaluate the usefulness of test cases in helping them construct code that meets expected outputs. Out of 42 respondents, a strong 31 students (73.8%) rated this feature a 5, while 6 students (14.3%) gave it a 4, showing widespread approval. Only 4 students (9.5%) rated it a 3, and just 1 student (2.4%) gave it a 2. No one rated it a 1. The resulting average rating was 4.60 out of 5, reflecting the effectiveness of NEUDev's test case system in guiding students toward correct program logic and output. This feedback suggests that the platform's test case functionality is highly beneficial for learning and debugging, supporting students in understanding the requirements and refining their code

accordingly.

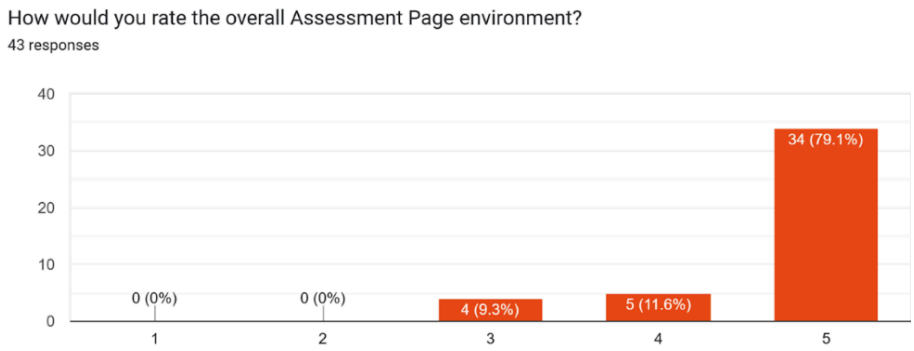


Figure 6.4 Student Ratings on the Overall Assessment Page Environment in NEUDev

Students were asked to rate their experience with the Assessment Page environment in NEUDev, where they conduct and complete their coding activities. From the 43 respondents, a dominant 34 students (79.1%) rated it a 5, while 5 students (11.6%) rated it a 4. Only 4 students (9.3%) gave it a 3, and no one rated it poorly (1 or 2).

The average rating achieved was 4.70 out of 5, making this one of the highest-rated features of the platform. These results reflect that the NEUDev assessment interface is not only functional but also well-received by its users. The positive feedback highlights its clean layout, smooth flow, and overall effectiveness in facilitating a focused and distraction-free coding environment for students.

To assess the user satisfaction and effectiveness of the NEUDev platform, a series of Google Forms surveys were conducted involving 42 to 43 student respondents. The overall feedback from the participants was overwhelmingly positive, highlighting the platform's usability, responsiveness, and functional features.

Most notably, the ***Assessment Page environment*** received the highest praise with an ***average rating of 4.70,*** followed closely by the ***class management features*** with ***4.76***, and the ***activities section*** with ***4.65***, indicating strong approval of NEUDev's core components.

The ***AI rubric grading system***, designed to provide automated and objective evaluation, was also well-received with an ***average score of 4.58***, suggesting that students trust the system's accuracy and usefulness. The i***ntegrated development environment (IDE)*** where students write code earned an average rating of ***4.60***, reflecting its intuitive and user-friendly design.Furthermore, the ***test case feature***, which helps students validate their outputs, was rated ***4.60***, demonstrating its effectiveness as a learning aid.

Collectively, these findings indicate that NEUDev successfully delivers a smooth, supportive, and effective assessment experience for students. The platform's user-centric features from activity delivery to grading automation are well-aligned with the learning needs of programming students at New Era University. The consistent high ratings across all categories validate NEUDev's role as a reliable and intelligent coding assessment platform.

System Usage

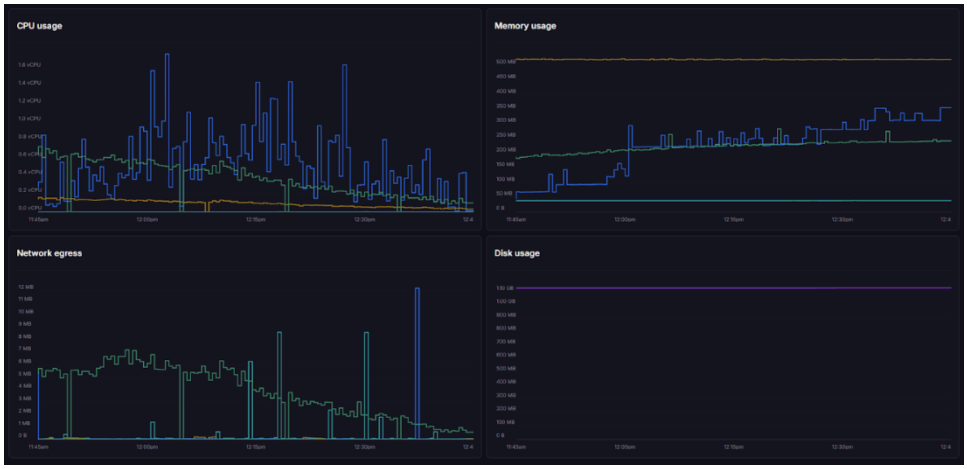


Figure 6.5 details the NEUDev system's resource utilization on May 14, 2025, over an approximately one-hour period (11:10 AM to 12:05 PM), reflecting the load from an examination with 33 concurrent students, and breaks down usage by key services: NEUDev_Compiler (blue), NEUDev_FE (green), MySQL database (orange), and overall disk usage (purple). In terms of CPU usage, the NEUDev_Compiler was highly dynamic with intermittent spikes peaking up to 1.7 vCPU due to compilation tasks, while the NEUDev_FE maintained a more sustained, moderate load between 0.3 and 0.8 vCPU from user interactions, and MySQL showed consistently low CPU use near 0.1 vCPU. For memory, NEUDev_Compiler usage increased in steps from around 25 MB to nearly 300 MB, and NEUDev_FE grew gradually from about 125 MB to over 200 MB.The MySQL line remained flat at 500 MB, likely representing its total allocated memor erating well m the NEUDe Dev_FE exhibite al. Disk usage ectively manag e most resource-intensive for peak CPU and network, and the NEUDev_FE showing consistent activity, all without signs of resource exhaustion.

Figure 6.6 NEUDev System Usage (May 15, 2025)

Figure 6.6 illustrates the NEUDev system's resource utilization on May 15, 2025, over an approximately one-hour window from 4:40 PM to 5:40 PM, during an examination session involving 24 students which commenced at 4:00 PM. The system components monitored are NEUDev_Compiler (blue), NEUDev_FE (green), MySQL database (orange), and disk usage (red). For the initial period until roughly 5:00 PM, all services exhibited minimal CPU and network activity. A significant event occurred just before 5:00 PM, where the NEUDev_FE (green) generated a large network egress spike exceeding 30 MB, after which it continued with frequent, smaller egress bursts. Post 5:15 PM, CPU usage by the NEUDev_FE (green) markedly increased, becoming the dominant consumer, peaking around 0.6-0.7 vCPU, while the NEUDev_Compiler (blue) showed only a slight rise in CPU activity, and MySQL (orange) remained low.

Memory usage for NEUDev_FE (green) saw a gradual increase to approximately 175 MB, and NEUDev_Compiler (blue) had a minor stepped increase towards the end; MySQL's memory line (orange), likely representing its total allocation, remained high but stable with a slight upward trend. Disk usage (red) was largely static, showing only a very small increase after 5:15 PM. This

pattern suggests an initial quiet phase, followed by a significant data transmission event primarily from the frontend, and a subsequent period of increased frontend processing activity, all handled without apparent resource strain.

In fulfillment of the sixth objective: **To evaluate the overall system using a rubric-based approach and assess the accuracy and reliability of the AI's automated grading**, the researchers used the testing dataset to evaluate the 5 criteria on how well the Gemini API with prompt engineering performs in evaluating the overall structure of the code. The results are discussed below:

Table 5.1 Comparison of Human and AI Grading Accuracy Across Performance Levels

Grade	Human Samples	AI Correct Prediction
Excellent	182	171
Good	133	122
Fair	93	73
Poor	40	29

The table 5.1 compares the number of submissions classified by human evaluators versus the number correctly predicted by the AI for each performance category. Out of 182 submissions labeled as "Excellent" by human evaluators, the AI correctly predicted 171, resulting in an accuracy rate of approximately 94% in this category. For "Good", the human total was 133, with the AI correctly identifying 122, achieving around 92% accuracy. In the "Fair" category, there were 93 human-labeled submissions, and the AI correctly predicted 73, which corresponds to about 78% accuracy. Lastly, for "Poor", the human count was 40, and the AI accurately identified 29, giving a prediction accuracy of 73%. These

results indicate that the AI aligns well with human judgment. It also suggests that the AI is functioning well in classifying and judging the students' submission code.

CHAPTER 5

CONCLUSION AND RECOMMENDATION

Conclusion

The development and use of NEUDev, an intelligent coding assessment platform for New Era University, have certainly overcome the shortcomings of traditional programming assessments and AI usage of students in code submission. NEUDev offers an improvement of both efficiency and integrity of coding assessments through integration with the use of modern web technologies such as ReactJS and Laravel, and AI tools such as CodeSpy to detect AI, Dolos to detect by similarity, and Gemini for rubric grading.

By integrating the Codex API via a WebSocket-based server and embedding the Monaco Editor with an XtermTerminal interface, NEUDev provides a seamless in-browser environment for authoring, compiling, and debugging Python, Java, and C# programs.

The integration of CodeSpy's AI Code Detection in the NEUDev was successfully achieved. As demonstrated in the results and discussion, the NEUDev coding assessment platform now has the backend services and frontend UI that uses the CodeSpy AI Code Detection. When the student successfully completes and submits his or her code in one of the assessments in the NEUDev the system effectively processes the source code and displays the AI Score to the teacher's activity records. This successful integration establishes the leveraging and utilization of AI in maintaining academic integrity in the

NEUDev platform. Since there will be AI code detection among submissions, students will now be forced to not rely on the usage of LLMs like ChatGPT, Gemini, Claude, etc., in the submission of their work. This will also lessen the student from being a vibe coder.

Instructors can efficiently review potential plagiarism cases through DOLOS similarity detection, which was implemented by developing Laravel backend services to batch student submissions to the DOLOS API and embedding the resulting links into the teacher's UI. This strengthens the platform's fairness guarantees.

A test-case-based scoring engine, augmented with a detailed rubric, was implemented through backend logic that evenly distributes item points, penalizes excessive code runs, and aggregates scores, while frontend components deliver immediate, criterion-level feedback to students, which reduces the teacher's workload in grading students.The results of the study demonstrate that, using a manually annotated synthetic dataset of 90 console programs (30 per language) as ground truth. The criteria Program Execution scored a high accuracy of 98% and the Time Taken scored 90%. On the other hand, the other criteria scored 86% for Design of Logic, 86% for the Coding Standard, and 80% for documentation.

These findings indeed confirm the system's ability to correctly distinguish human and AI generated code, hence enhancing the academic integrity of

programming evaluations. Overall, the system represents significant advancements for academic assessments for programming education.

Recommendations

The following recommendations and improvements are proposed for the future improvement of NEUDev, an intelligent coding assessment platform developed for New Era University.

First, is to increase the programming languages as the system currently only supports Java, C# and Python. This is to give a wider range of programming instruction in taking programming assessments. This will allow the platform to be more flexible across various computer related courses. Moreover, is to improve the behavioral assessment monitoring, as currently the system only monitors the alt tab switch during exam mode. Additional behavioral assessment monitoring features will enhance the assessment of the credibility of the students to support academic integrity.

Additionally, is to improve the test case-based scoring framework, as currently the system divides the points of the test cases based on the item point. It should allow teachers to assign a specific point for each test case, which gives flexibility and emphasis on the weight of the test cases.To improve the accuracy performance of the NEUDev's rubric system, we suggest to use an LLM model that is finetuned in coding dataset. Another is that in the future work, we might finetune an LLM that is based on code to learn the structured rubric instead of generating it from prompt engineering. So that, we can fully enhance the

evaluation of the coding structure of the students' submissions. We also expect in the finetuned model to answer in a structured way already so that it will be an ease to get the data and use it for academic purpose. Aside from the inputs of the rubric system: Question, Code, Allotted Time, and Time Taken, we suggest to put a new parameter "Expected Answer." The new parameter will serve as one of the bases of the rubric system in evaluating the student code submission. In this we can ensure that the submission of the student is not in a brute force manner instead it follows the right structure and pattern. Since the AI Code Detection uses a pretrained model by the CodeSpy, we recommend that in the future researches in the New Era University, there will be a study on deep learning about the AI Code Detection.

This will significantly lower the cost of the production of the NEUDev platform for it will now use its own model that detect AI generated codes. We also recommend that the NEU student should also develop similarity detection tool among code submissions that can be integrated to the NEUDev coding assessment platform. So that, we will not rely on external systems that will be connected to the NEUDev.Furthermore, it is recommended that the NEUDev be made open source, as this will engage collaboration and innovation of the system. This initiative would enable developers to actively participate in the improvement of the system. Specifically, this can introduce additional features that are related to machine learning, deep learning and AI integration of the system.

Lastly, consistent updates of the current features of the system should be maintained to ensure its effectiveness and relevance. These recommendations aim to maximize the potential of NEUDev as a significant solution for programming education.

REFERENCES

Amoozadeh, M., Nam, D., Prol, D., Alfageeh, A., Prather, J., Hilton, M., Srinivasa Ragavan, S., & Alipour, M. A. (2017). Transferring arithmetic decision procedures (on Z) to alternative representations. In *StudentAI interaction: A case study of CS1 students*. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Arias, R., Martinez, G., Cáceres, D., & Garces, E. (2024). Limitations and benefits of the ChatGPT for Python programmers and its tools for evaluation. In A. Abraham & P. P. Wang (Eds.), *Cybernetics and control theory in systems (CSOC 2024)* (pp. 171–194). Springer. https://doi.org/10.1007/9783031703003_12

Aung, S. T., Funabiki, N., Syaifudin, Y. W., Kyaw, H. H. S., Aung, S. L., Dim, N. K., & Kao, W. C. (2021). A proposal of grammarconcept understanding problem in Java programming learning assistant system. *Journal of Advances in Information Technology, 12*(4), 342–350. https://doi.org/10.12720/jait.12.4.342350

Baqar, M., & Khanda, R. (2024, September 9). The future of software testing: AIpowered test case generation and validation [Preprint]. arXiv. https://arxiv.org/abs/2409.05808

Becker, B. A. (2019). *A survey of introductory programming courses in Ireland*. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '19)*. ACM. https://doi.org/10.1145/3304221.3319752

Bers, M. U. (2019). Coding as another language: A pedagogical approach for teaching computer science in early childhood. *Journal of Computers in Education*. https://doi.org/10.1007/s40692019001473

BlueOptima. (2024). *Code Author Detector Overview*. Retrieved April 24, 2025, from https://www.blueoptima.com/top4aisourcecodedetectortoolsforenterprises/

Bukhari, S. A. (2024). *Issues in detection of AIgenerated source code* (Master's thesis, University of Calgary). University of Calgary Institutional Repository. https://ucalgary.scholaris.ca/items/e1dcccfdd1b24b76af3c2794b917d21

Bulla, L., Midolo, A., Mongiovì, M., & Tramontana, E. (2024). EXCODE: A robust and explainable model to detect AIgenerated code. *Information, 15*(12), 819. https://doi.org/10.3390/info15120819

Carvalho Filho, R. M. de, Soares Neto, C. de S., & dos Santos, D. V. (2024). *Expanding code assessment: A qualitative feedback system for beginning students*. In *Proceedings of the 16th International Conference on Computer Supported Education (CSEDU 2024)*. SCITEPRESS. https://www.scitepress.org/Papers/2024/127416/127416.pdf

Charan, T. R., Satyanarayana, I., Krishna, M. J., Kumar, A. S., Siddartha, G. S., & Srija, H. (2023). Coding assessment portal. *Journal of Emerging Technologies and Innovative Research, 10*(6), 123–135. https://doi.org/10.12345/JETIR.2023.06.012

Chen, H. M., Chen, W. H., Hsueh, N. L., Lee, C. C., & Li, C. H. (2017). *ProgEdu An automatic assessment platform for programming courses*. In *Proceedings of the 2017 International Conference on Applied System Innovation* (pp. 173–176). Sapporo, Japan. https://www.researchgate.net/publication/318694393_ProgEdu__an_automatic_assessment_platform_for_programming_courses

Chen, S., Zilles, C., & Xie, T. (2024). *Cheating method change and learning loss in an intro to CS course.* University of Illinois at Urbana-Champaign. https://zilles.cs.illinois.edu/papers/chen_chatgpt_cheating_l_s_2024.pdf

Combéfis, S. (2022). Automated code assessment for education: Review, classification and perspectives on techniques and tools. *Software, 1*(1), 3–30. https://doi.org/10.3390/software1010002

Demirok, B., & Kutlu, M. (2024). AIGCodeSet: A new annotated dataset for AIgenerated code detection [Preprint]. arXiv. https://arxiv.org/abs/2412.16594

Dunder, N., Lundborg, S., Wong, J., & Viberg, O. (2024). *Kattis vs ChatGPT: Assessment and evaluation of programming tasks in the age of artificial intelligence*. In *Proceedings of the 24th Koli Calling International Conference on Computing Education Research* (Article 16, pp. 1–10). ACM. https://doi.org/10.1145/3636555.3636882

Ferguson, R., & D'Souza, D. (2023). *Identifying Plagiarised Programming Assignments with Detection Tools. International Journal of Information and Education Technology,* 13(6), 571-578. https://files.eric.ed.gov/fulltext/EJ1385258.pdf

Gambo, I., Abegunde, F.J., Gambo, O., Ogundokun, R. O., Babatunde, A. N., & Lee, C.C. (2024). GRADAI: An automated grading tool for code assessment and feedback in programming course. *Education and Information Technologies*. https://doi.org/10.1007/s10639024132185

Girardin, M. (2023, December 11). What is HackerRank? *Forage*. https://www.theforage.com/blog/skills/whatishackerrank

Girardin, M. (2023, December 15). What is LeetCode? *Forage*. https://www.theforage.com/blog/skills/whatisleetcode

Gupta, A., Hathwar, D., & Vijayakumar, A. (n.d.). *Introduction to AI chatbots*. Dayananda Sagar College of Engineering. https://pdfs.semanticscholar.org/f5f4/746acffef08df37f184cb6acc0505362ea9b.pdf

Huang, L., Zhang, H., Li, R., Ge, Y., & Wang, J. (2020). AI coding: Learning to construct error correction codes. *IEEE Transactions on Communications, 68*(1), 26–39. https://doi.org/10.1109/TCOMM.2020.8890904

Islam, T., Khan, R., & Smith, J. (2024). A comparative analysis of programming language preferences among computer science and noncomputer science students. *European Journal of Theoretical and Applied Sciences, 2*(3), 900–912. https://doi.org/10.59324/ejtas.2024.2(3).70

JaagravSeal. (n.d.). *Codex7 API documentation*. RapidAPI. Retrieved April 24, 2025, from https://rapidapi.com/jaagravseal03/api/codex7

Joy, M., Cosma, G., Yau, J. Y. K., & Sinclair, J. (2022). *A case study on contract cheating in computer science education*. IEEE Transactions on Education, 65(1), 34-41.doi:10.1109/TE.2021.3082568

Joy, M., Griffiths, N., & Boyatt, R. (2022). *The BOSS online submission and assessment system* [Technical report]. University of Warwick. https://www.dcs.warwick.ac.uk/reports/csrr415.pdf

Jukiewicz, M. (2023). The future of grading programming assignments in education: The role of ChatGPT in automating the assessment and feedback process. *Journal of Computer Science Education, 32*(2), 123–137. https://doi.org/10.13140/RG.2.2.22103.85924

Ketkar, A., Tsantalis, N., & Dig, D. (2020). *Understanding type changes in Java*. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)* (pp. 1–13). ACM. https://doi.org/10.1145/3368089.3409725

Khoirom, S., Sonia, M., Laikhuram, B., Laishram, J., & Singh, T. D. (2020). Comparative analysis of Python and Java for beginners. *International Research Journal of Engineering and Technology (IRJET), 7*(8), 4384–4396. https://www.irjet.net

Kong, S. C., & Liu, B. (2020). A performancebased assessment platform for developing computational thinking concepts and practices: EasyCode. *IEEE Transactions on Learning Technologies*. https://tc.computer.org/tclt/10110920200201001/

Kumar, M. (2025). AIRA: AIpowered code review & bug detection system. *International Journal of Scientific Research and Engineering Management, 9*(3), 1–9. https://doi.org/10.55041/IJSREM42592

Krill, P. (2024). Python scores its highest rating in Tiobe index. *InfoWorld*. https://www.infoworld.com/article/3481502/pythonscoresitshighestratingintiobeindex.html

Leinonen, J. L., Niemi, T., & Koistinen, J. (2024). *Detecting ChatGPTgenerated code submissions in a CS1 course using machine learning models*. In *Proceedings of the North Carolina State University's 2024 Teaching and Education Conference* (Vol. 1). ACM. https://doi.org/10.1145/3626252.3630826

Li, B., Jiang, J., Ma, L., Huang, Y., & Wu, X. (2024). Mitigating university students' reliance on ChatGPT. In L. Chang, Y. Zhang, & S. Lee (Eds.), *Proceedings of the 2024 8th International Seminar on Education, Management and Social Sciences (ISEMSS 2024)* (Advances in Social Science, Education and Humanities Research, Vol. 867, pp. 245–251). Atlantis Press. https://doi.org/10.2991/9782384762972_31

Li, Y., Zhang, Q., & Chen, L. (2024). *CodeVision: Vision models are effective zeroshot detectors for AIgenerated code* [Preprint]. arXiv. https://arxiv.org/abs/2501.03288

Li, Y., Tan, T., & Xue, J. (2019). Understanding and analyzing Java reflection. *ACM Transactions on Software Engineering and Methodology, 28*(2), Article 7. https://doi.org/10.1145/3295739

LuxtonReilly, A., Craig, M., Carter, J., Denny, P., Fincher, S., Robins, A., & Sheard, J. (2022). Constructing and validating a model of student plagiarism. *Journal of Computer Assisted Learning, 38*(1), 241–254. https://doi.org/10.1111/jcal.12633

Maertens, A., Bogaerts, S., Deroover, K., Joosen, W., Piessens, F., & De Meuter, W. (2022). Dolos: Languageagnostic plagiarism detection in source code. *Journal of Computer Assisted Learning, 38*(4), 1041–1055. https://doi.org/10.1111/jcal.12662

Maranga, J. J. A., Matugas, L. K. J., Lim, J. F. W., & Sta. Romana, C. L. C. (2019). *CodeChum: A smart IDE for teaching and learning programming*. *IADIS International Journal on WWW/Internet, 17*(1), 93–110. https://www.iadisportal.org/ijwi/papers/2019171107.pdf

Misiejuk, K., Kallisa, R., & Scianna, J. (2024). Augmenting assessment with AI coding of online student discourse: A question of reliability. *Computers and Education: Artificial Intelligence, 6*, 100216. https://doi.org/10.1016/j.caeai.2024.100216

Mishra, A., Chandel, A., & Motwani, D. (2020). Extended MABAC method based on divergence measures for multicriteria assessment of programming languages with intervalvalued intuitionistic fuzzy sets. *Granular Computing, 5*, 97–117. https://doi.org/10.1007/s41066020001053

Millam, A., & Bakke, C. (2024). Coding with AI as an assistant: Can AI generate concise computer code? *Journal of Information Technology Education:*

Innovations in Practice, 23, 113–126. https://jite.org/documents/Vol23/JITEIIPv23Art009Millam10355.pdf

Munisamy, M., Md Osman, S. Z., & Sanmugam, M. (2024). *Code, click, learn: A systematic review of online assessment tools in 21st century programming education* [Technical report]. Universiti Sains Malaysia. https://www.researchgate.net/publication/379478754

Oedingen, M., Engelhardt, R. C., Denz, R., Hammer, M., & Konen, W. (2024). ChatGPT code detection: Techniques for uncovering the source of code [Preprint]. arXiv. https://doi.org/10.3390/ai5030053

Oedingen, M., Reinhard, C., & Konen, W. (2024). *ChatGPT Code Detection: Techniques for Uncovering the Source of Code* [Preprint]. arXiv. https://arxiv.org/abs/2405.15512

Originality.AI. (2024). *ZeroGPT review*. Originality.AI. https://originality.ai/blog/zerogptaicontentdetectorreview

Pan, W. H., Chok, M. J., Wong, J. L. S., Shin, Y. X., Poon, Y. S., Yang, Z., Chong, C. Y., Lo, D., & Lim, M. K. (2024). Assessing AI detectors in identifying AIgenerated code: Implications for education. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSESEET '24)* (pp. 1–11). ACM. https://doi.org/10.1145/3639474.3640068

Pan, W. H., Wang, X., & Li, Y. (2024). Assessing AI detectors in identifying AIgenerated code [Preprint]. arXiv. https://arxiv.org/abs/2401.03676

Robinson, P. E., & Carroll, J. (2017). An online learning platform for teaching, learning, and assessment of programming. In *2017 IEEE Global Engineering Education Conference (EDUCON)* (pp. 1259–1264). IEEE. https://doi.org/10.1109/EDUCON.2017.7942900

Rogers, S., Smith, A., & Lee, J. (2023). ACCE: Automatic coding composition evaluator [Preprint]. ResearchGate. https://www.researchgate.net/publication/261959765_ACCE_Automatic_coding_composition_evaluator

Rytilahti, J., & Kaila, E. (2024). How easy is it to cheat? Solving programming exercises automatically with AI. In *Proceedings of the 20th International CDIO Conference* (pp. 1–10). CDIO. https://cdio.esprit.tn:9080/documents/1717971995327218_b_Final.pdf

Schildt, H. (2021). *C# 3.0: The complete reference*. McGrawHill/Osborne. http://117.250.119.200:8080/jspui/bitstream/123456789/1372/1/C%23%203.0%20%20The%20Complete%20Reference%20%20Herbert%20Schildt.pdf

Serth, S., Staubitz, T., Teusner, R., & Meinel, C. (2021). CodeOcean and CodeHarbor: Autograder and code repository. In *Seventh SPLICE Workshop at SIGCSE: CS Education Infrastructure for All III* (2022). https://github.com/openHPI/codeharbor

Staubitz, T., Klement, H., Teusner, R., Renz, J., & Meinel, C. (2016). CodeOcean – A versatile platform for practical programming exercises in online environments. In *2016 IEEE Global Engineering Education Conference (EDUCON)* (pp. 314–323). IEEE. https://doi.org/10.1109/EDUCON.2016.7474573

Sobral, S. R. (2021). *Advances in digital science*. Springer Nature Switzerland AG. https://link.springer.com/book/10.1007/9783030717827

Sokač, M., Novak, T., & Petrovic, B. (2023). Motivation – manual grading is timeconsuming and inconsistent; automated systems (APASs) provide objective, efficient grading with timely feedback. *Future Education*, 7(2), 41. https://www.mdpi.com/25044990/7/2/41

Strozanski, P. (2023). *Anticheat for programming courses: Detecting AIgenerated source code* (Master's thesis, Aalto University). Aaltodoc. https://aaltodoc.aalto.fi/server/api/core/bitstreams/49248f2ee4f14af2941e946e214b6325/content

Suh, H., Tafreshipour, M., Li, J., Bhattiprolu, A., & Ahmed, I. (2024, November 6). An empirical study on automatically detecting AIgenerated source code: How far are we? [Preprint]. arXiv.org. https://arxiv.org/abs/2411.04299

Tendulkar, S. (n.d.). *Automated test case generation and defect detection through generative AI solutions* [Preprint]. ResearchGate. Retrieved April 24, 2025, from https://www.researchgate.net/publication/385763899

Tiong, L. C. O., Lee, Y., Lim, K. L., & Lee, H. J. (2024). Advancing online assessment integrity: Integrated misconduct detection via internet protocol analysis and behavioral classification. *IEEE Access, 12*, Article 10613749. https://doi.org/10.1109/ACCESS.2024.3434608

Vakaliuk, T. A., Spirin, O. M., Lobanchykova, N. M., Martseva, L. A., Novitska, I. V., & Kontsedailo, V. V. (2021). Features of distance learning of cloud technologies for the organization educational process in quarantine. *Journal of Physics: Conference Series, 1840*(1), 012051. https://doi.org/10.1088/17426596/1840/1/012051

Vetrivel, S. C., Arun, V. P., Ambikapathi, R., & Saravanan, T. P. (2025). Automated grading systems: Enhancing efficiency and consistency in student assessments. In *Adopting artificial intelligence tools in higher education* (1st ed., p. 21). CRC Press. https://doi.org/10.1201/97810034703043

Wang, H., Zhao, Y., & Liu, X. (2024). *GPTSniffer: A finetuned detector for AIgenerated code* [Preprint]. arXiv. https://arxiv.org/abs/2307.09381

Wang, H., Liu, J., Kang, J., Yin, W., Sun, H., & Wang, H. (2020). Feature envy detection based on BiLSTM with selfattention mechanism. In *2020 IEEE International Conference on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social*

Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom). https://doi.org/10.1109/ISPABDCloudSocialComSustainCom51426.2020.0008

Winter. (2024). 5 AIpowered content detectors to help identify AIgenerated text. *dev.to*. https://dev.to/joywinter90/5aipoweredcontentdetectorstohelpidentifyaigeneratedtext1e99

Xu, Z., & Sheng, V. S. (2024). Detecting AIgenerated code assignments using perplexity of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(21), IAAI24. https://ojs.aaai.org/index.php/AAAI/article/view/30361

Yang, W., Hu, Z., Lin, L., Liu, J., & Duan, L.Y. (2024). Coding for intelligence from the perspective of category [Preprint]. arXiv. https://arxiv.org/pdf/2407.01017

Zinovieva, I. S., Petrova, O. V., & Ivanov, A. A. (2021). The use of online coding platforms as additional distance tools in programming education. *Journal of Physics: Conference Series, 1840*(1), 012029. https://doi.org/10.1088/17426596/1840/1/012029

Zhou, L., & Singh, R. (2023). *On the Feasibility of Detecting AI-Generated Code in Introductory Programming Assignments*. arXiv preprint arXiv:2201.07406. https://arxiv.org/abs/2201.07406

Zhou, Y., Yang, H., & Chen, Z. (2024). Effects of ChatGPT on Programming Test Performance and Efficiency. arXiv preprint arXiv:2309.16697. https://arxiv.org/abs/2309.16697

ONLINE REFERENCES

CodeSpy. (n.d.). *What does it do?* CodeSpy. Retrieved April 24, 2025, from https://codespy.ai/pages/whatdoesitdo

CodeSpy. (2024). *CodeSpy: AIgenerated code detector*. CodeSpy. https://codespy.ai

CodeSpy. (2024). *Official website*. CodeSpy. https://code.aidetector.pro/

Codeleaks. (2024). *Official overview*. Codeleaks. https://www.100ai.tools/tool/codeleaks

Google. (n.d.). *Google AI Studio*. Retrieved April 24, 2025, from https://aistudio.google.com/welcome